

ALGORITHMS

❑ ADT – Abstract Data Type

CLRS: 10

❑ 1-d : List, Queue, Stack

❑ Implementation, complexity

❑ 2-d: Rooted Trees

❑ Binary Trees

See appendix B.5 Trees)

❑ Traversals (pre-, in-, post-order)

❑ Heap (aka Priority Queue)

CRLS: 6

❑ BST

CRLS: 12

❑ Red-Black Trees

CRLS: 13 (note: CLRS has no AVL trees)

❑ Huffman Encoding

CRLS: 16.3

I-D CONTAINERS: LISTS, QUEUES, STACKS

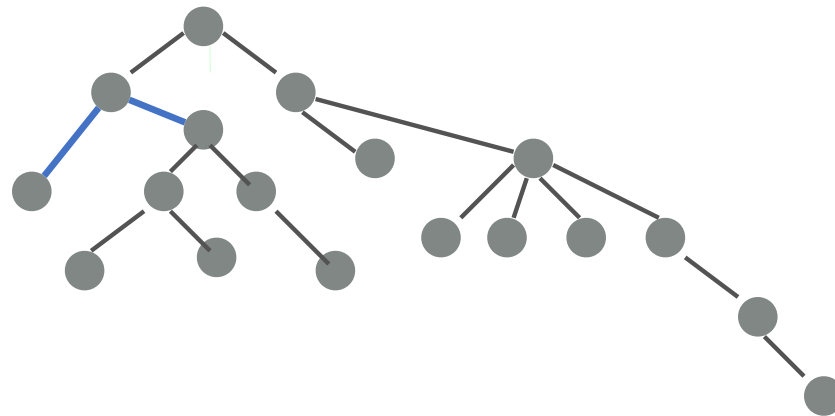
- **Abstract Data Types (ADT):** data type (class) with ops (methods).
 - ◆ Examples: Int. (0,1,...,Maxint). All 2 by 2 real matrices. IEEE floats, etc.
- ◆ **List of size N:** $A_0, A_1, \dots, A_{N-1}$,
 - ◆ Each element A_k has a unique position in the list, elements can be arbitrarily complex, elements ordered
 - ◆ Useful Methods: addHead, deleteHead, addEnd, deleteEnd, findEntry
 - ◆ Implementation: either as arrays or as linked lists—changes complexity. findEntry is $O(n)$.
- **Queue (First-In, First-Out) is a list with primary methods:**
 - ◆ Enqueue(item): addEnd & Dequeue(item): deleteHead
 - ◆ Scheduling, graph traversals, printer queues, I/O streams, ...scheduling, graph traversals, priority queues, routing, printer queues, I/O streams
- **STACK (Last In, First Out) is a list with methods:**
 - ◆ push(item): addHead, item = Top: readHead, item = pop(): readHead and deleteHead
 - ◆ Scheduling, graph traversals, printer queues, I/O streams, recursive functions, undo/redo in editors², ...

Problem of Interest: Search

- Problem: finding an element among a group of elements
 - Data structure issue: what is a good way of storing that data so search is fast?
- Issues to consider:
 - Will the data change? How often? How?
 - Do we search for arbitrary elements by value? By relative value (e.g. largest, smallest, 5th largest?) By order of insertion (e.g. last in, first out (LIFO) or first in, first out (FIFO)?
- One approach: Sorted lists
 - With array implementation, many steps are fast
 - Find k-th largest: $O(1)$; find x: Binary search (bisection) $O(\log(n))$
 - Find median: $O(1)$
 - But insert, delete, or change_value are $O(n)$ — 1 step of insertion sort, and form list is $O(n\log(n))$

INTRODUCTION TO TREES

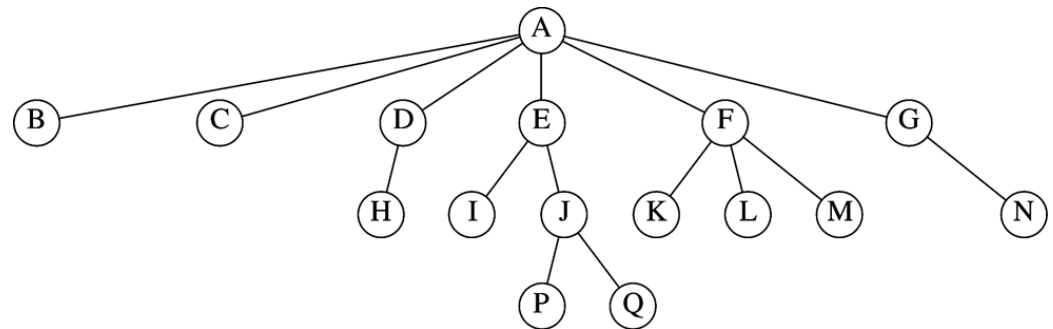
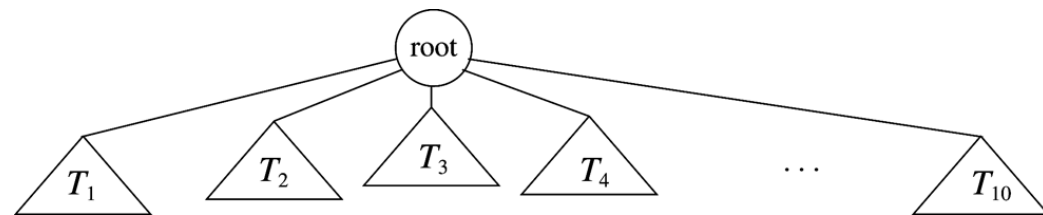
- *Trees: inheritance, partial ordering, execution graphs, ...*
- *A tree is a special kind of Graph $G(E, V)$*
- *E = “edges/arcs” connecting V = “vertices/nodes”*



- *A tree is **Connected**, **Acyclic**, **Undirected** $G(E, V)$*
- *We don't know yet what a Graph is...we will use this later in the course.*

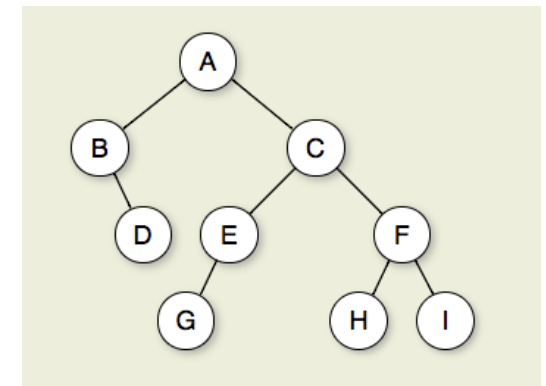
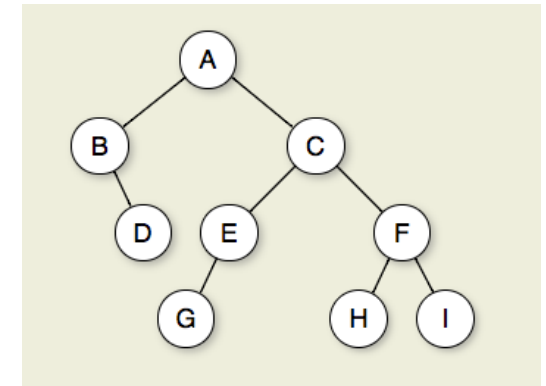
Rooted Trees

- Root node with rooted trees as children
 - Every node except the root node has a exactly 1 parent
- Important concepts
 - Node: a structure with data and pointers to children
 - Edge: a link from a node to a node that is one of its children
- Sample applications: File systems, compilers, domain tables, ...



Language of Rooted Trees

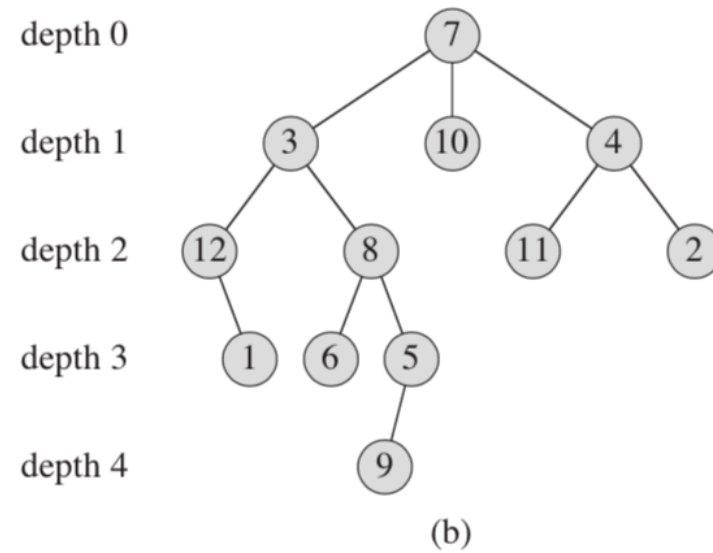
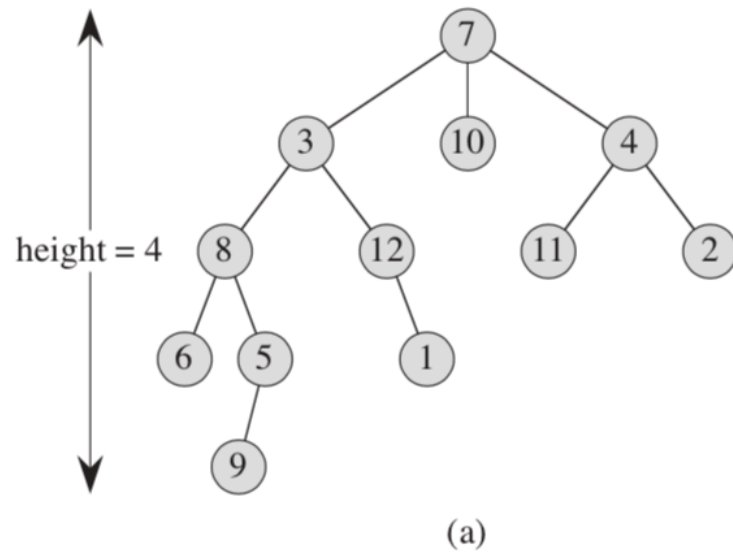
- **Node:** an element of the tree. Contains data, pointers to other nodes
- **Root:** The entry node of the tree. Has no parent
- **Edge:** the link from a node to a child node
- **Child(X):** A node that is linked from node X, has X as parent
- **Parent(X):** The node in the tree that has X as its child
- **Sibling(X):** a node that has the same parent as X
- **Leaf:** A node with no children nodes
- **Internal node:** has at least one child
- **Path(n_1, n_k):** a sequence $n_1, n_2, \dots, n_k$ so n_{i+1} is child of n_i
- **Depth(n):** number of edges on path from root to n.
- **Height:** maximum depth of any leaf
- **Ancestor:** n ancestor of m if there is path from n to m
- **Descendant:** m is a descendant of n if there is path from n to m



Height vs Depth of “nodes”

B.5 Trees

1177

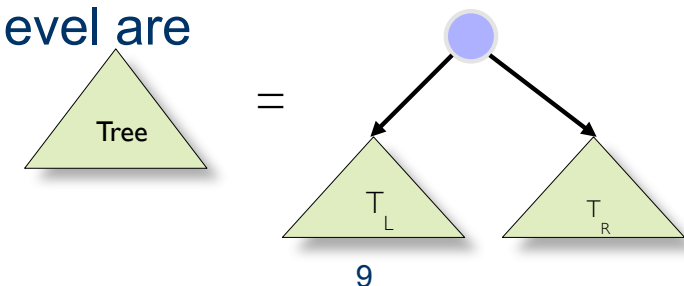
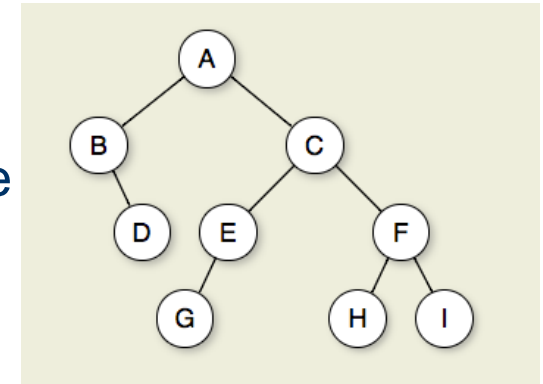


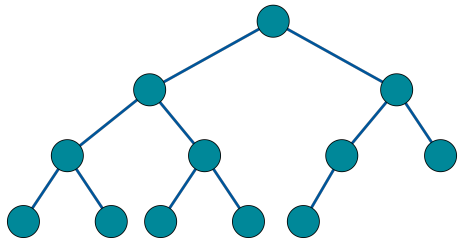
PARTIAL ORDER

- Set: $S = \{a, b, c, \dots\}$
- Partial order is a binary relationship R on $S \times S$: A subset of $S \times S$, where $a R b$ is True
- Properties:
 - ◆ Reflexive: $a R a$ is True
 - ◆ Anti-symmetric: $a R b$ and $b R a \rightarrow a = b$
 - ◆ Transitive: $a R b$ and $b R c \rightarrow a R c$
- Parent descendant relationship is partial ordering among nodes in a tree.

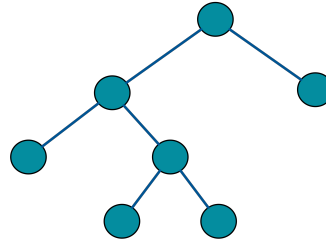
Special Case: Binary Trees

- Binary Tree: Every node has at most 2 children (0, 1 or 2)
- Recursive definition: A binary tree is null, or a single node with a Right and Left Child that is a binary tree!
- Special cases:
 - **Full Binary Tree**: Every node has 0 or 2 children
 - **Perfect Binary Tree**: all interior nodes have two children and all leaves have the same depth
 - **Complete Binary Tree**: every level, *except possibly the last*, is completely filled, and all nodes in the last level are as far left as possible

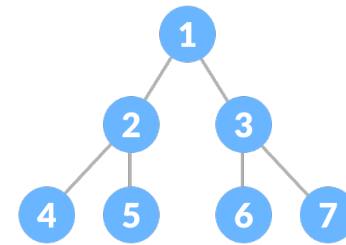




Complete binary tree



Full binary tree



Perfect binary tree

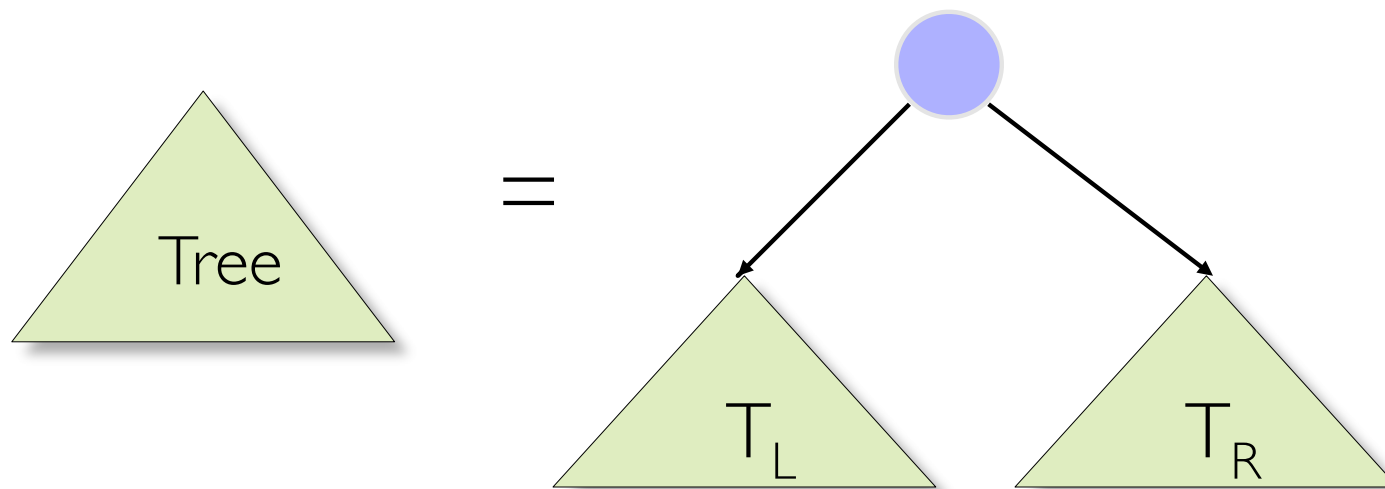
- Simple counting results:
 - Full binary tree: number of leaves = number of internal nodes + 1
 - In non-empty binary tree, number of null children = number of nodes in trees+1
 - In perfect binary tree, number of nodes is $2^{\text{height}+1} - 1$
 - For n nodes, height = $\log_2(n + 1) - 1$

DEFINITIONS FOR BINARY TREES

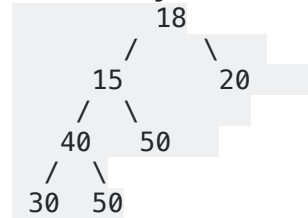
- ◆ Full Tree: 0, 2 children,
- ◆ Complete Tree: Consecutive nodes (aka Heap),
- ◆ Perfect Tree: Complete and full last row.
- Full Tree Theorem: # of leaves: $L(N) = (N+1)/2$ for N nodes
- Perfect Tree with H levels (height or depth)
- Nodes in Perfect k-way tree : $N(H) = (k^{H+1}-1)/(k-1) \rightarrow 2^{H+1} - 1$
- Execution Tree
- Traversals: in-, pre-, post-order.

BINARY TREE: RECURSIVE DEFINITION

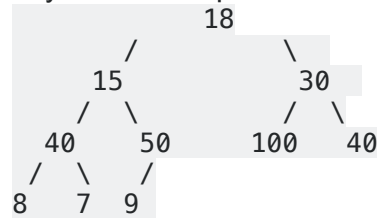
- A binary tree is null or a single node with a Right and Left Child that is a binary tree!
(Useful for organizing recursive algorithms on binary trees.)



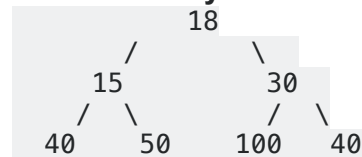
Full Binary Tree: A Binary Tree is full if every node has 0 or 2 children. Following are examples of a full binary tree.



Complete Binary Tree: A Binary Tree is complete Binary Tree if all levels are completely filled except possibly the last level and the last keys as left as possible.



Perfect Binary Tree: A Binary tree is Perfect Binary Tree in which all internal nodes have two children and all leaves are at same level.

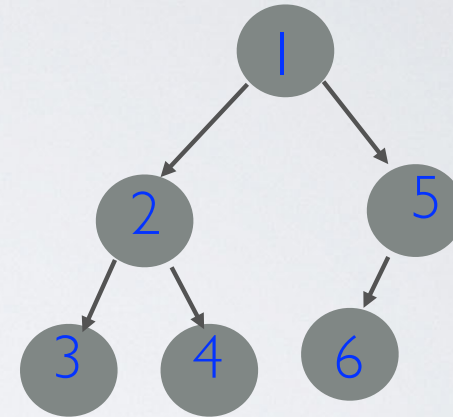


BINARY TREE TRAVERSALS

■ Preorder: Print [Tree]{
 Print root;
 Print Tree[LeftTree];
 Print Tree:[RightTree];
 }

■ Inorder: Print [Tree]{
 Print Tree[LeftTree];
 Print root;
 Print Tree:[RightTree]
 }

■ Postorder: Print [Tree]{
 Print Tree[LeftTree];
 Print Tree:[RightTree]
 Print root;
 }

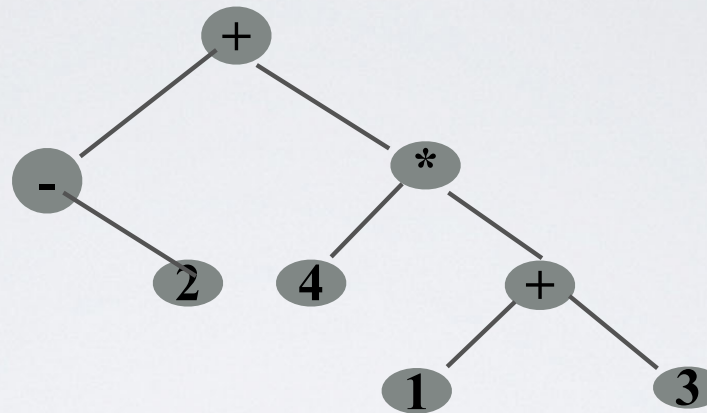


Pre: 1→2→3→4→5→6

In: 3→2→4→1→6→5 sort on BST

Post: 3→4→2→6→5→1

Expression Trees



Preorder: + - 2 * 4 + 1 3 (Lisp, Scheme) (+ (- 2) (* 4 (+ 1 3)))

In order: -2 + 4 * (1 + 3) (C, C++, Java) Standard precedence

Postorder: 2 - 4 1 3 + * + (HP calculator, PS, Forth)

Binary Search Tree: left $\leq$ root < right

- in order traversal gives sorted list
- easy to search

see https://en.wikipedia.org/wiki/Binary_expression_tree

DIMENSIONS OF A PERFECT TREE

- Perfect Tree (all levels filled) with H levels:

(Height: $H = \lfloor \log_k(n) \rfloor$ for k-array tree)

- # nodes: $N(H) = 1 + k + k^2 + k^3 + \dots + k^H = (k^{H+1}-1)/(k-1)$

(binary tree: $N = 1 + 2 + 2^2 + 2^3 + \dots + 2^H = 2^{H+1} - 1$)

- total Depth: = sum of all depths of nodes = $0 + k + 2k^2 + \dots + Hk^H$

$$= k \frac{d}{dk} \frac{k^{H+1} - 1}{k - 1} = \frac{(H+1)k^{H+1}}{k-1} - \frac{k(k^{H+1} - 1)}{(k-1)^2}$$

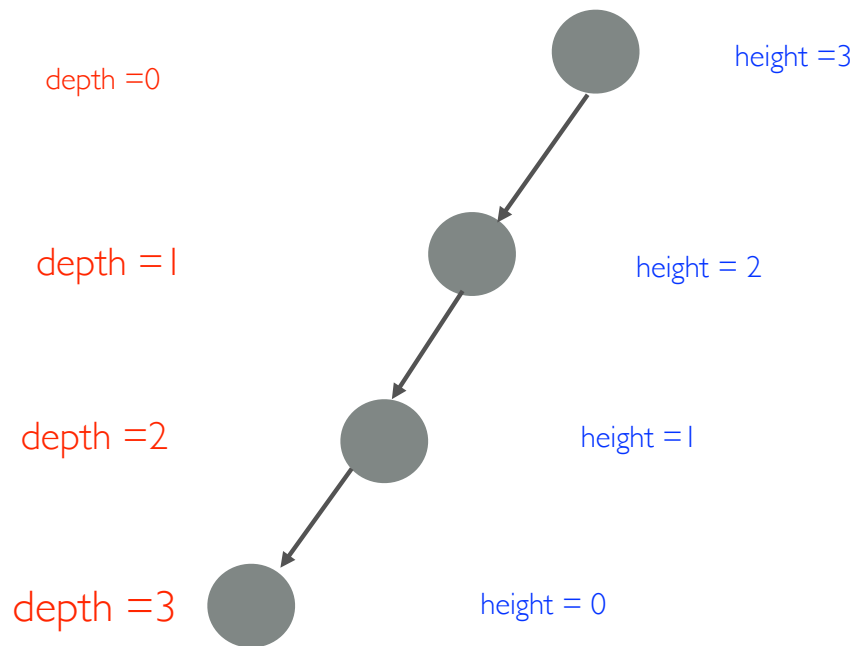
→ (binary tree) $2(H+1)2^H - 2(2^{H+1}-1) = (H-1)N(H) + H + 1$

- total Height: $T_H(N) + T_D(N) = H N$ (each h + d = H)

$$T_H = H N - T_D = N - H - 1 \quad (\text{binary tree})$$

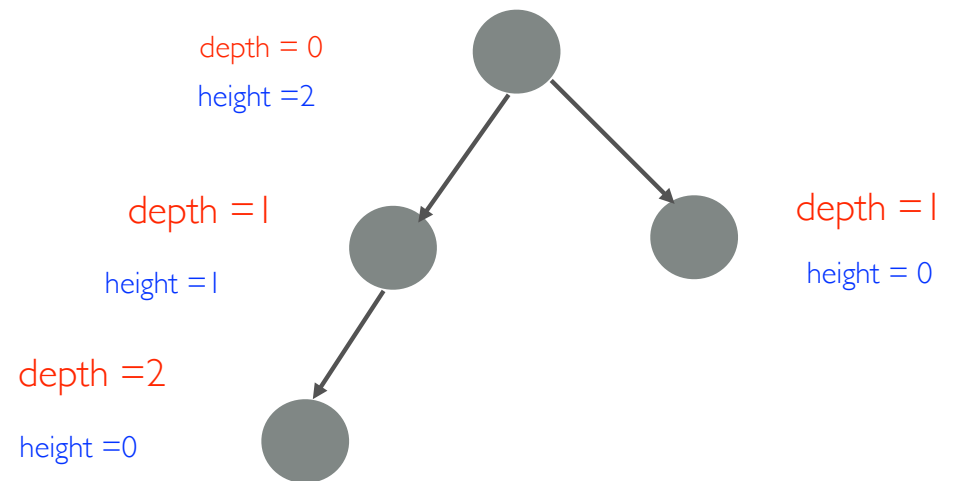
Fun Examples

$H = 3, N = 4$



$$Th(N) + Td(N) = 6 + 6 = 12 = 3 \cdot 4 = H \cdot N$$

$H = 2, N = 4$



$$Th(N) + Td(N) = 3 + 4 = 7 < 2 \cdot 4 = H \cdot N$$

Binary Heap = Binary tree implemented with arrays

- Basic Heap ADT: Ch. 6, CLRS

- ◆ *Data is $a[i]$, $i = 1, \dots, N$ (ROOT = 1, LABEL 0) (FORTRAN or MATLAB, not C, Java, Python, ...)*
- ◆ *Methods: Insert (key), Delete(key), DeleteMin, Build and Sort*
- ◆ *Heap property (min-heap) Key of parent $\leq$ key of children*

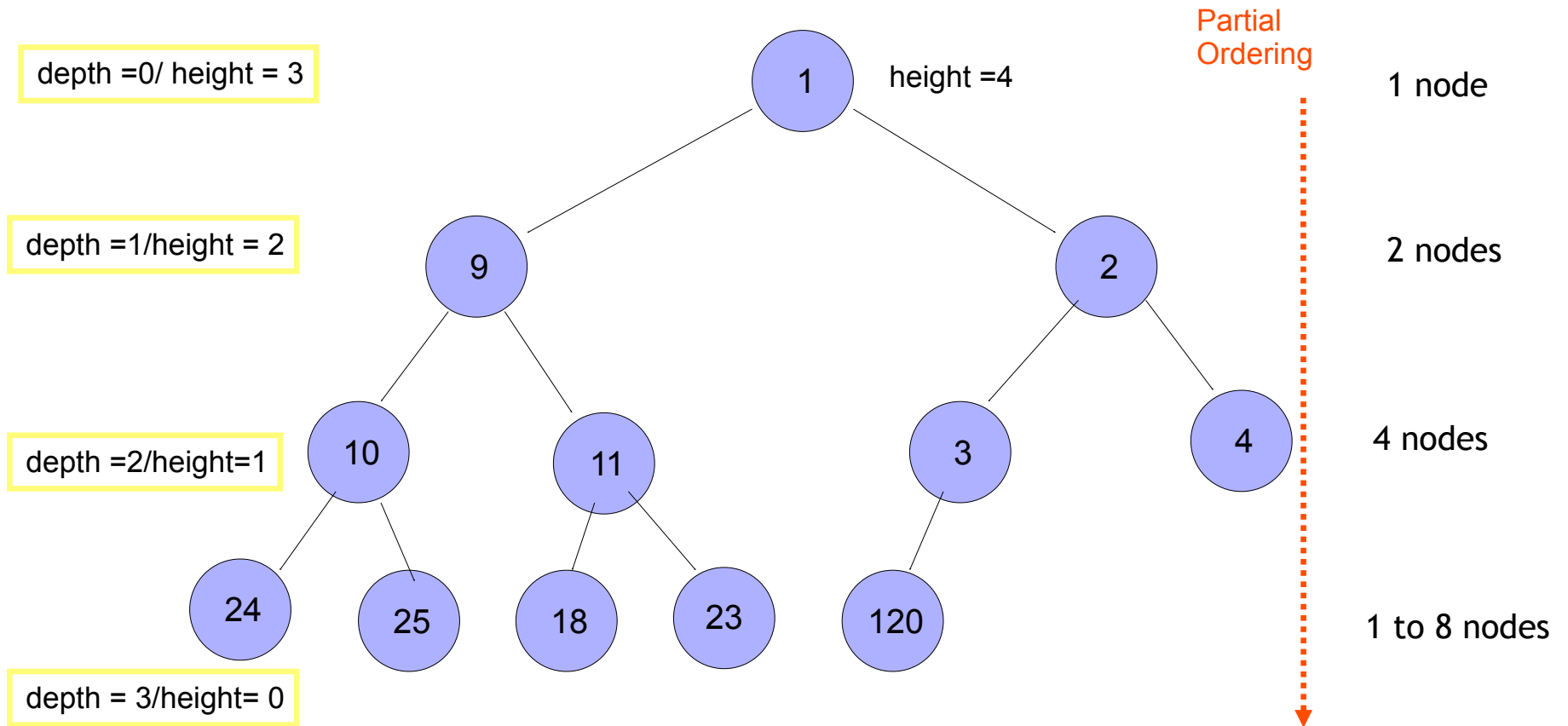
- Q: When is a tree an array? A: complete tree

- ◆ *Parent $a[i] \rightarrow a[2i] = \text{left child} \ \& \ a[2i+1] = \text{right child}$*
- ◆ *Child $a[j]$: $\rightarrow a[j/2] = \text{parent}$ (integer division).*

- Build Heap is $O(N)$ by bottom up, DeleteMin is $O(\log(N)) \rightarrow$

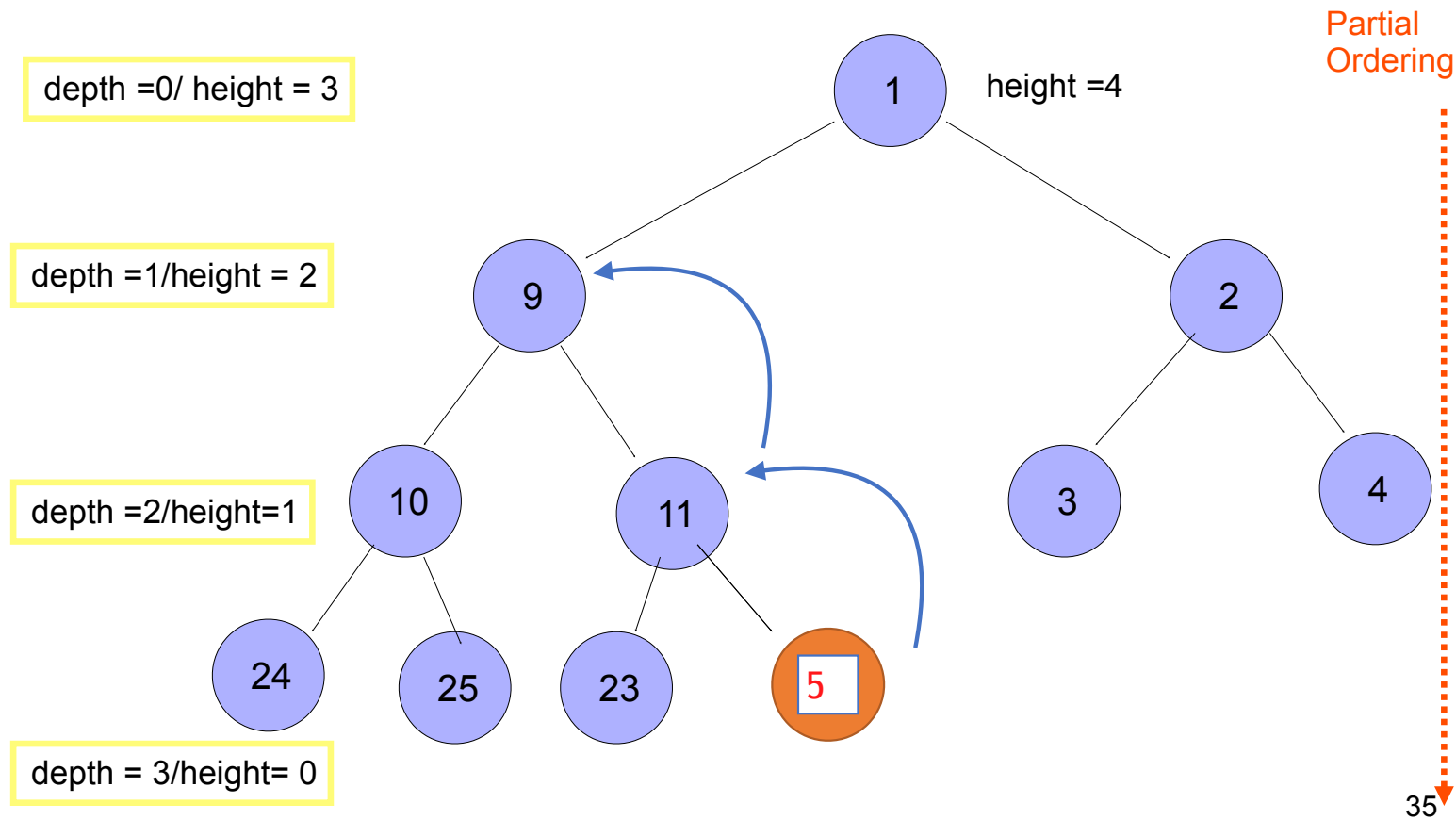
- ◆ *Heap sort by deleting min over and over is $O(N \log(N))$.*

Min Heap Order



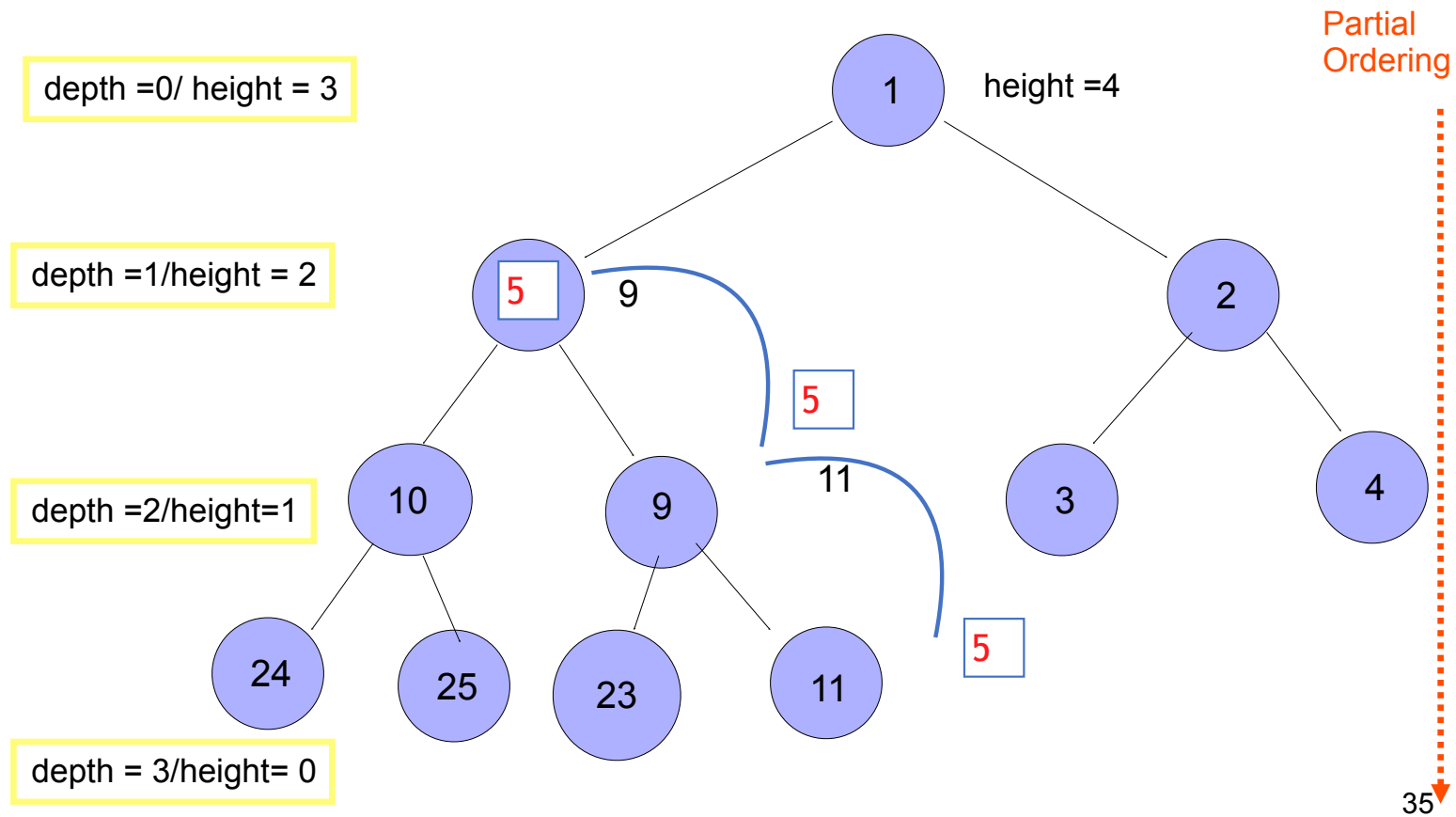
MAX NUMBER OF NODES $N = 1 + 2 + 4 + \dots + 2^H = 2^{H+1} - 1$ and Total height $H = O(\log_2(N))$

Insert 5: Insertion sort on path to root

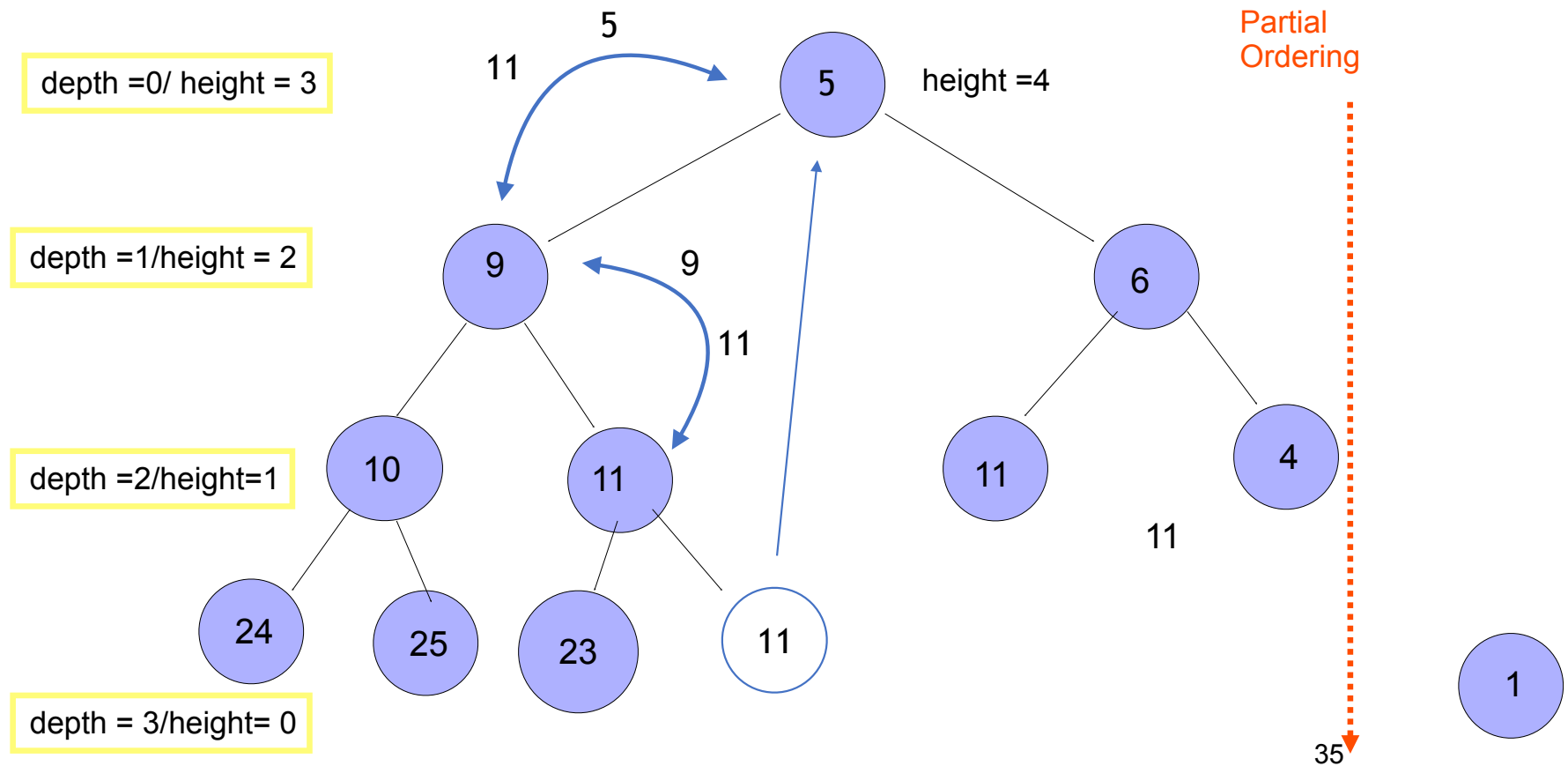


Insert at end, up heap using insertion sort on path to root to restore heap property

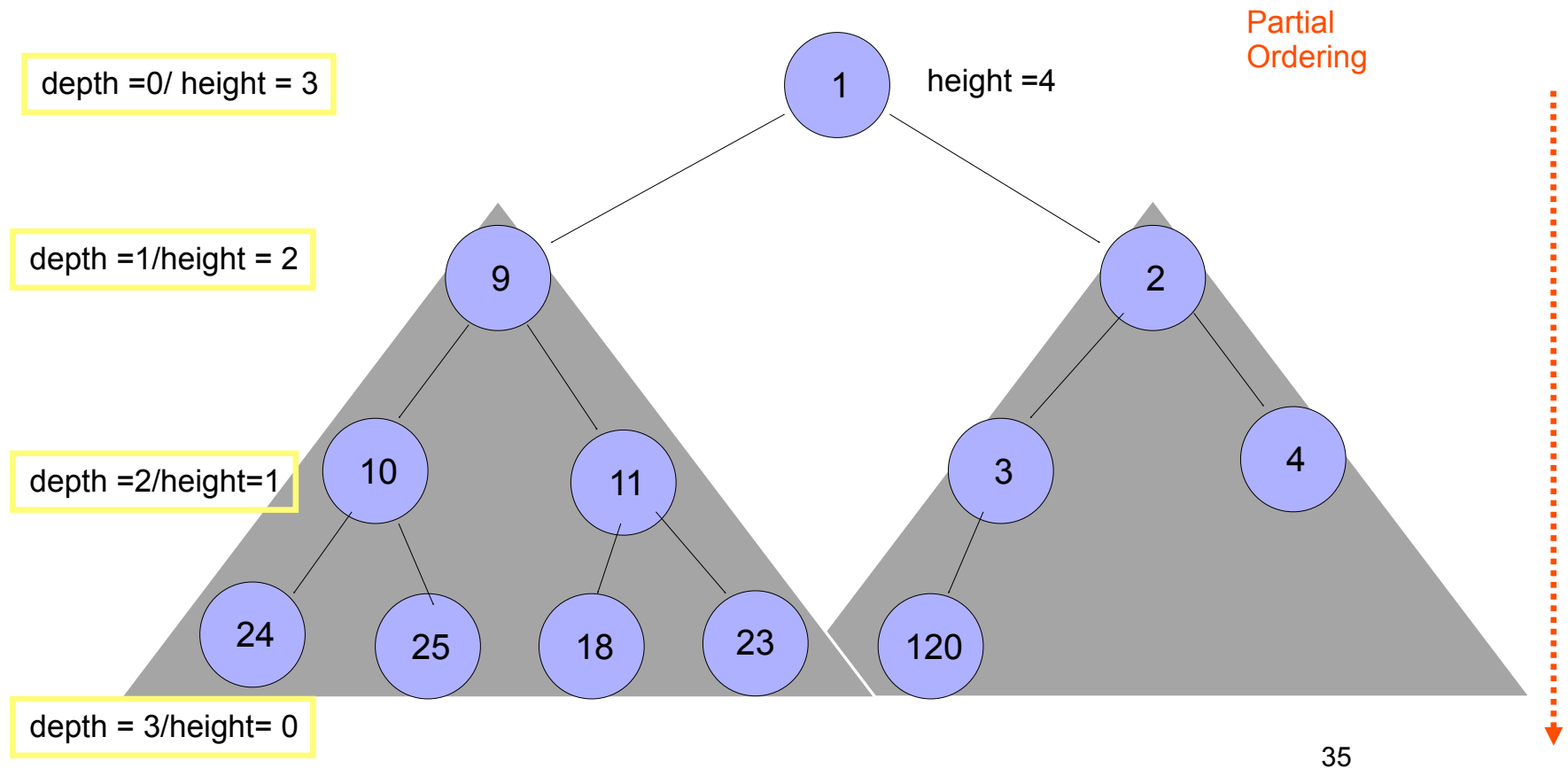
Insert 5: Insertion sort on path to root



Delete min (1): Swap last element to root, downheap

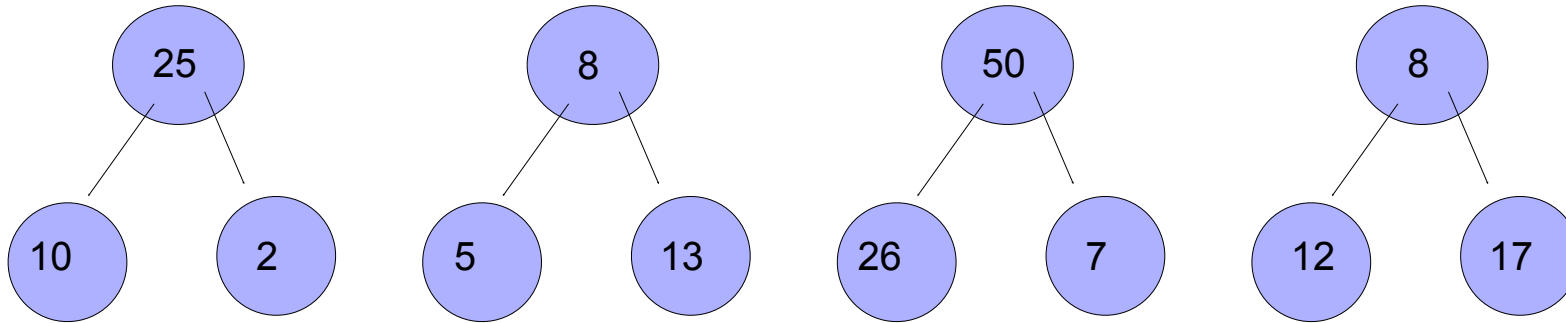


Recursive: Heap = Root + Left and Right Heap



Bottom up Heapify in $O(N)$

- Take Array as is and then heapify the bottom at most $2^{(H-1)}$ pairs



$$T(N) \leq 2^{H-1} + 2 * 2^{H-2} + 3 * 2^{H-3} + \dots + (H-1) * 1 = 2^H \sum_{n=1}^{H-1} n(1/2)^n$$

Do the sum for large total height H by for $x = 1/2$

$$\sum_{n=0}^{H-1} nx^n = x \frac{d}{dx} \sum_{n=0}^{H-1} x^n = x \frac{d}{dx} \frac{1-x^H}{1-x} \simeq 1/(1-x)^2 = 4$$

or Solve the Recursive relation with $N = 2^H$

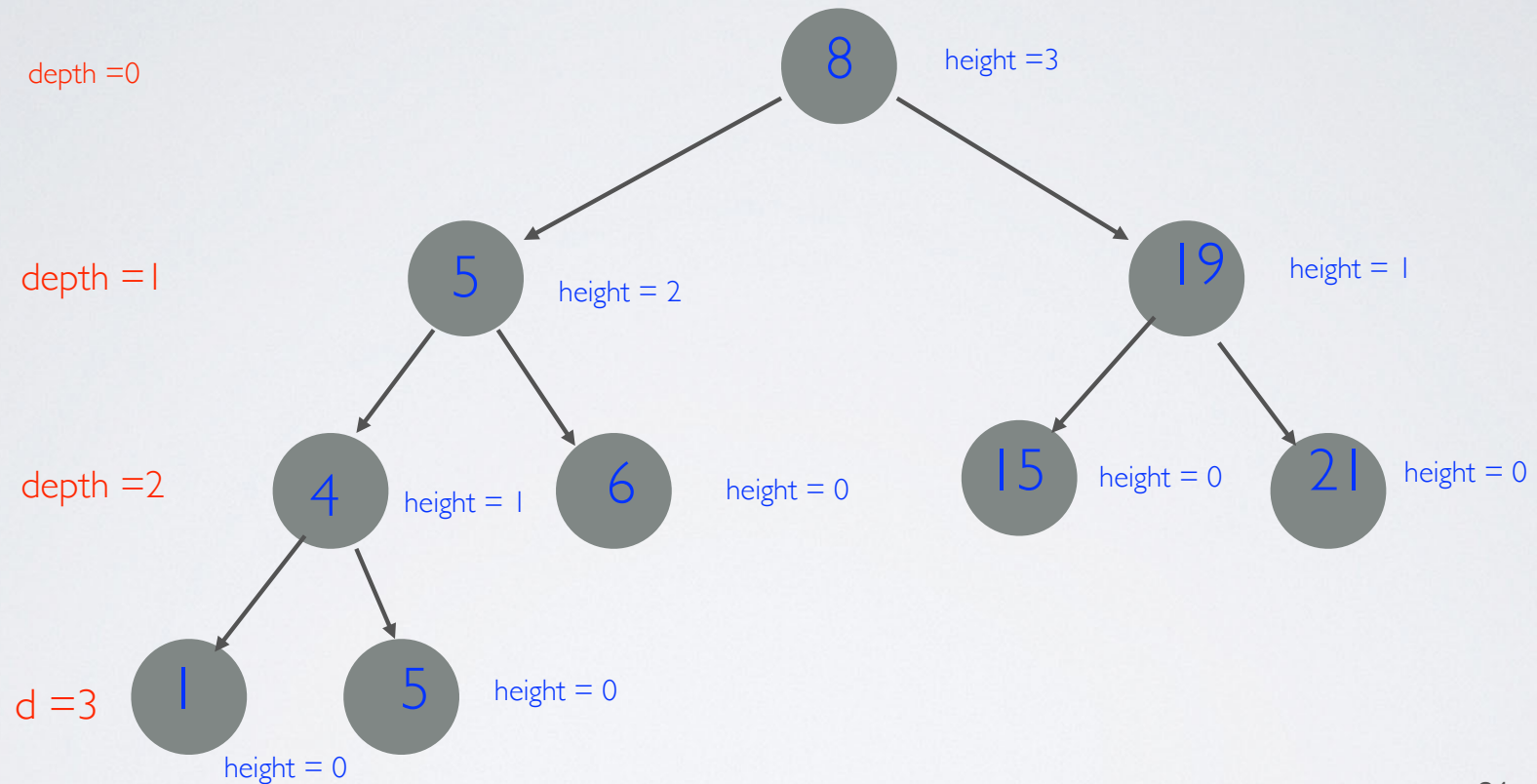
$$T(H) = 2T(H-1) + c_0H \text{ with the guess } T = c_1 2^H + c_2 H$$

Complexity of Heap Operations

- Height of heap: $O(\log(n))$
- insert: $O(\log(n))$
 - 2.607 comparisons on average, i.e., $O(1)$ on average
- deleteMin: $O(\log(n))$
- decreaseKey: Find key, change value, upheap: $O(\log(n))$
- increaseKey: Find key, change value, downheap: $O(\log(n))$
- remove: Find key, swap last element into key position, downheap: $O(\log(n))$
- buildHeap: $O(n)$
- findMinimum: $O(1)$

- Merge two heaps: $O(n)$ —> link two arrays and buildHeap

AVL: BST WITH $|H_L - H_R| = 0, 1$

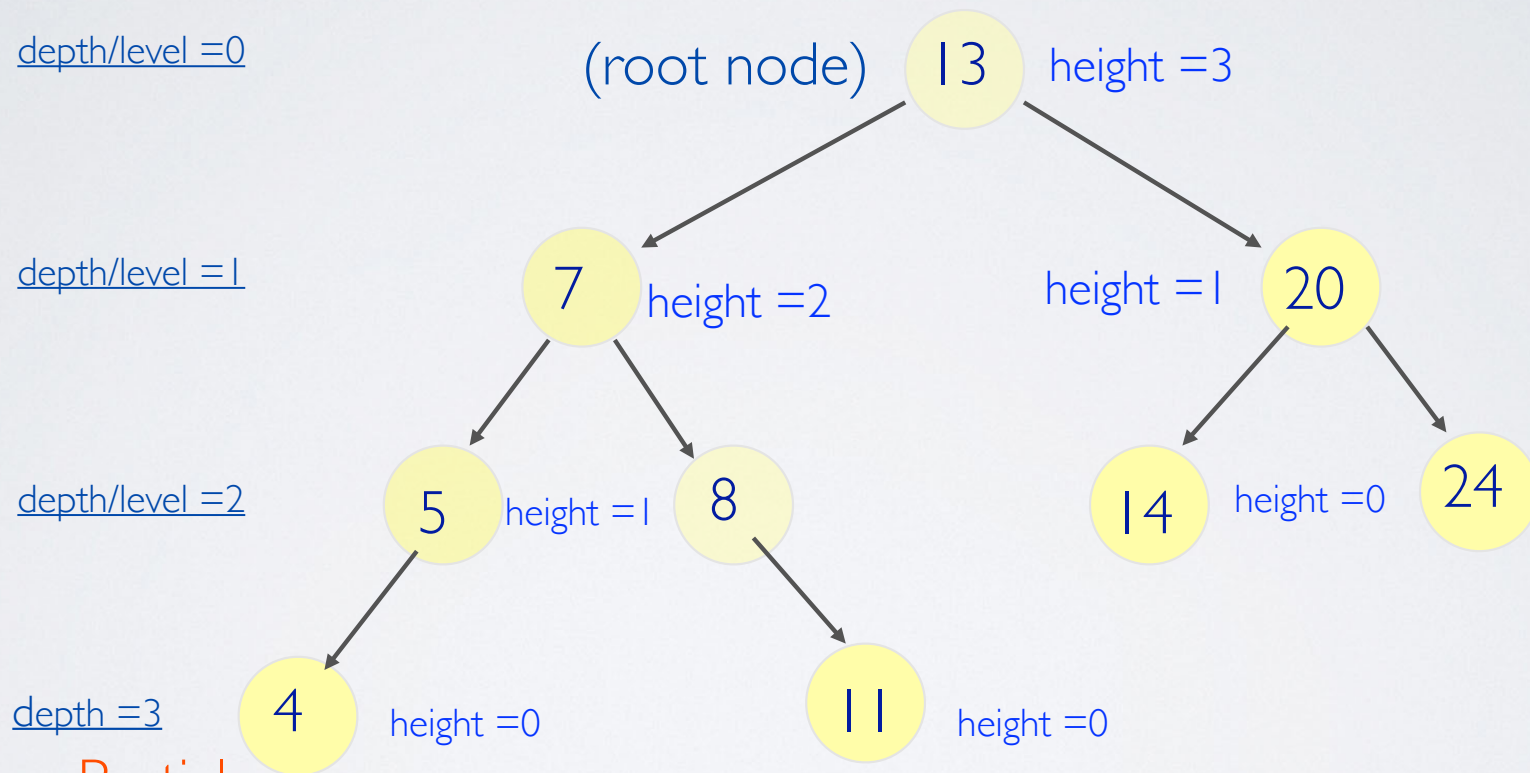


<https://www.geeksforgeeks.org/avl-with-duplicate-keys/>

SEARCH TREES

- *BST tree Recursive definition*
 - ◆ *Insertion and Deletion*
- *AVL tree balance:*
 - ◆ *Insertions: single (zig-zig) and double (zig-zag) rotations.*
 - ◆ *Lazy Deletion*
- *Red/Black Tree*

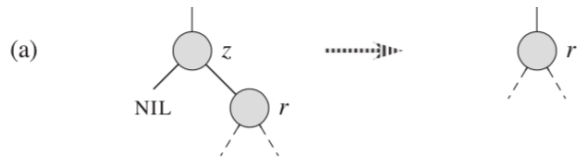
BINARY SEARCH TREES



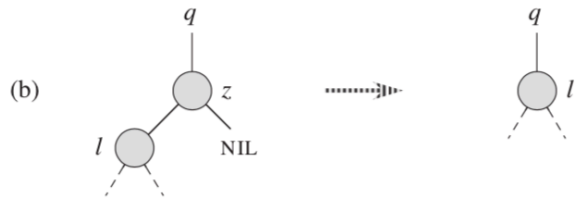
Partial
Ordering

BINARY SEARCH TREE: BST

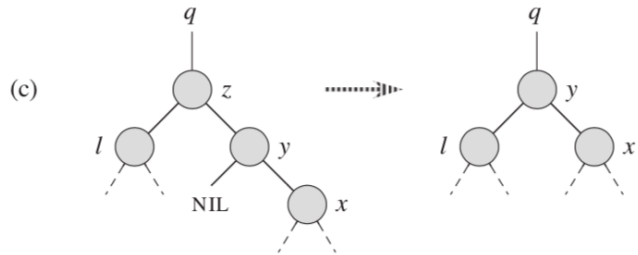
1. BST is a Binary Tree with keys stored in each node.
2. The key (K_0) in each node is: greater or equal to all keys in T_L , the Left subtree ($K_{\text{left}} \leq K_0$) less than all keys in T_R , the Right subtree ($K_0 < K_{\text{Right}}$)
3. The BST defines a partial ordered set --- as you move down to the left/right the keys decrease/increase.
4. Insert new K_{new} push down to subtree Left/Right if $K_{\text{new}} \leq / > K_0$.
5. If K_0 has only one child, replace K_0 by its child. Otherwise, Delete K_0 and replace by SMALLEST key in T_R , the right subtree of K_0 . The right child of SMALLEST replaces SMALLEST in tree.



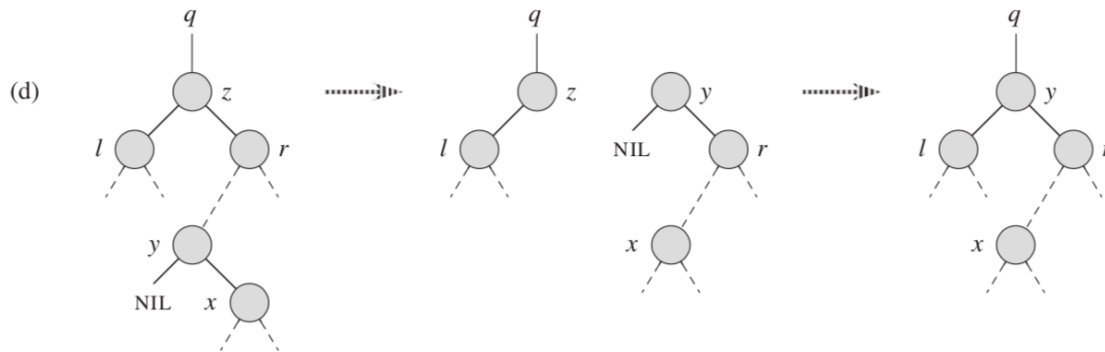
Right Only Child



Left Only Child



R & L Twins



Right to Left Rotation

Note $y < x < r$ and $l < z < r, y, x$

BST DELETE Z

AVERAGE TOTAL DEPTH OF BST

$$\begin{aligned} \blacksquare T_D(N) &= \frac{2}{N}[T_D(0) + T_D(1) + T_D(2) + \cdots + T_D(N-1)] + c(N-1) \\ T_D(x) &\simeq \frac{2}{x} \int_0^x T_D(x) + c(x-1) && \text{SAME AS QUICK SORT!} \\ xT_D(x) &\simeq 2 \int_0^x T_D(x) + c(x^2 - x) \\ &\Rightarrow T_D(x) + x \frac{dT_D(x)}{dx} = 2T_D(x) + c(2x-1) \\ \frac{dT_D(x)}{dx} &\simeq T_D(x)/x + 2c \\ &\Rightarrow T_D(x) = 2cx \log(x) \end{aligned}$$

◆ Solution: $T_D(N) = \Theta(N \log(N))$

See Average of Quick Sort Sec ²⁰7.7.5 (p 278)

Binary Search Tree: Comments

- In-order traversal yields sorted list!
- Operations
 - Insert, Delete, Find: $O(h)$, where h is height of tree
 - Find minimum, maximum: $O(h)$
- Average Height: $O(\log(n))$, but worst case height: $O(n)$...trees can be unbalanced
- Problem: h can be $O(n)$, for n nodes! Trees can be unbalanced...
- Solution: Use height-balanced binary search trees: h is $O(\log(n))$
 - AVL trees (not that interesting, not in use, not in textbook)
 - Red-Black trees (used in many applications: `java.util.TreeMap` , `java.util.TreeSet`, `java.util.HashMap`; C++ STL: `map`, `set`, `multimap`, `multiset`; Linux kernel: completely fair scheduler, ...)

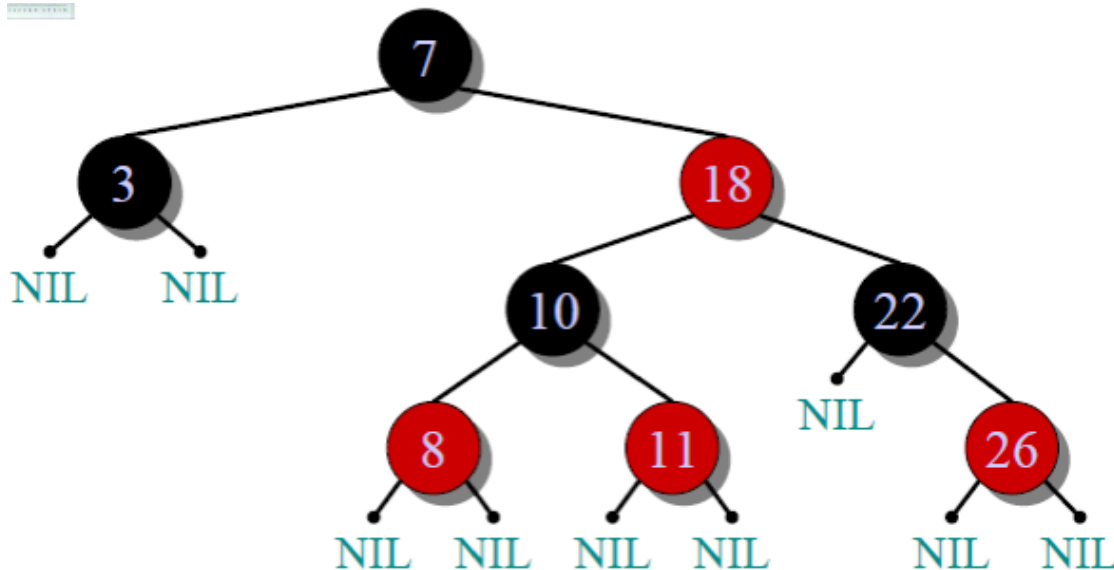
Binary Search Tree: Comments

- In-order traversal yields sorted list!
- Operations
 - Insert, Delete, Find: $O(h)$, where h is height of tree
 - Find minimum, maximum: $O(h)$
- Average Height: $O(\log(n))$, but worst case height: $O(n)$...trees can be unbalanced
- Problem: h can be $O(n)$, for n nodes! Trees can be unbalanced...
- Solution: Use height-balanced binary search trees: h is $O(\log(n))$
 - AVL trees (not that interesting, not in use, not in textbook)
 - Red-Black trees (used in many applications: `java.util.TreeMap` , `java.util.TreeSet`, `java.util.HashMap`; C++ STL: `map`, `set`, `multimap`, `multiset`; Linux kernel: completely fair scheduler, ...)

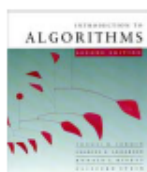
Red Black Trees (Ch. 13, CLRS)

- Binary search tree
 - Each node is colored **red** or **black**.
 - The root is black
 - If a node is red, its children must be black
 - Every path from a node to a NULL reference must contain the same number of black nodes
 - Null references are colored black
- Data structure requires an extra one-bit color field in each node

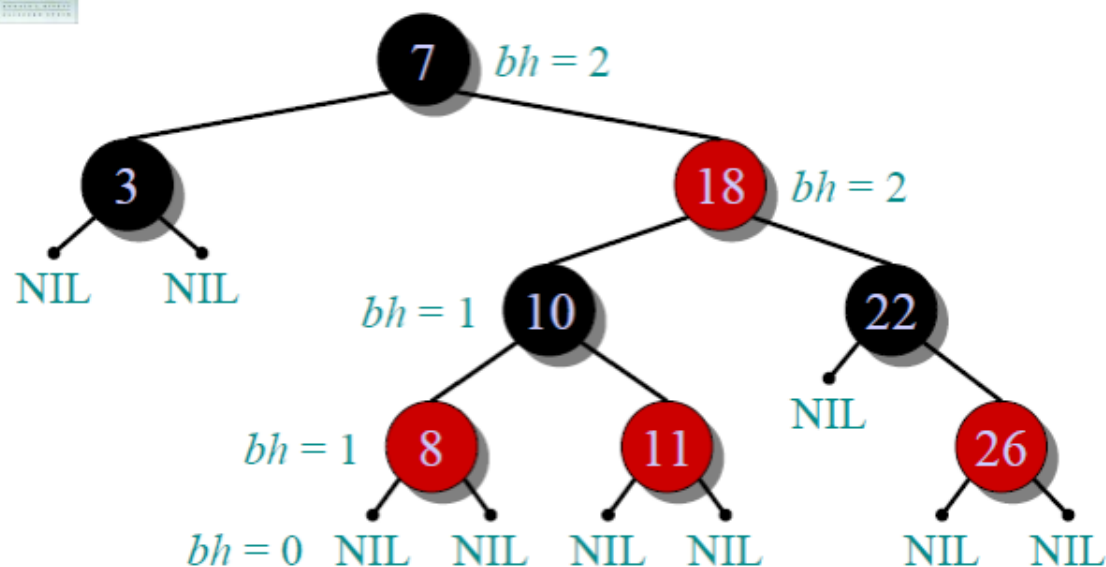
Example of Red Black Trees



- Each node is colored red or black.
- Root is black
- No red node with red parent
- Null Nodes are black
- Every path from a node to a leaf contains the same number of black nodes
—> **Black height**: does not include current node but includes null nodes

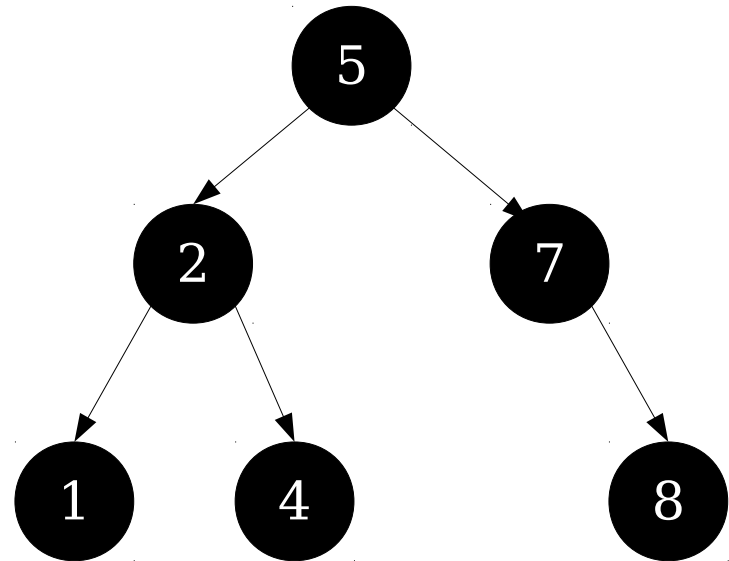
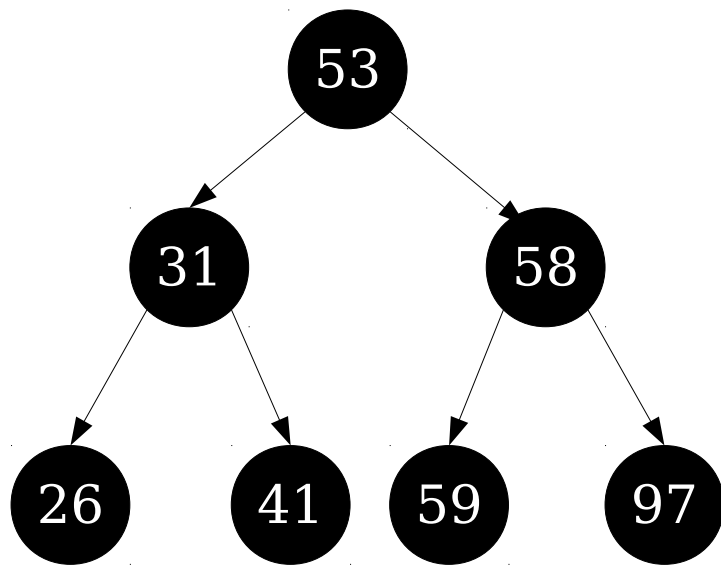


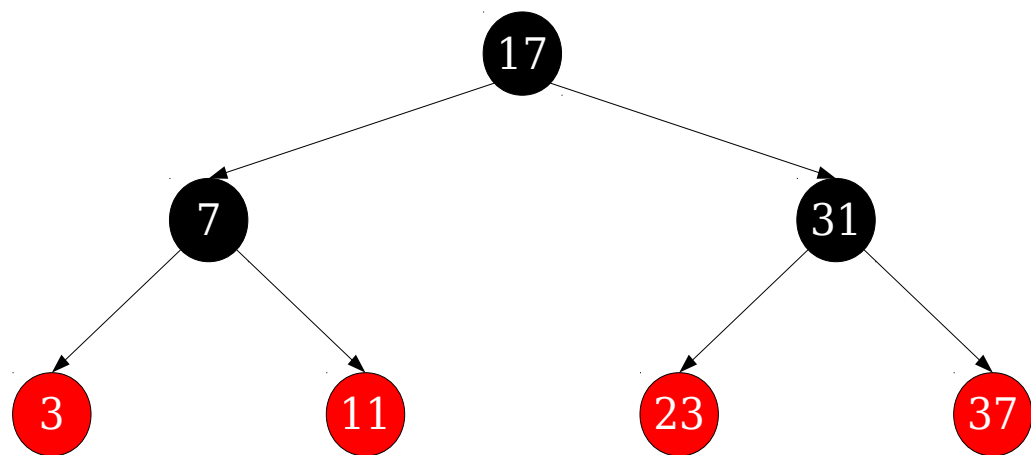
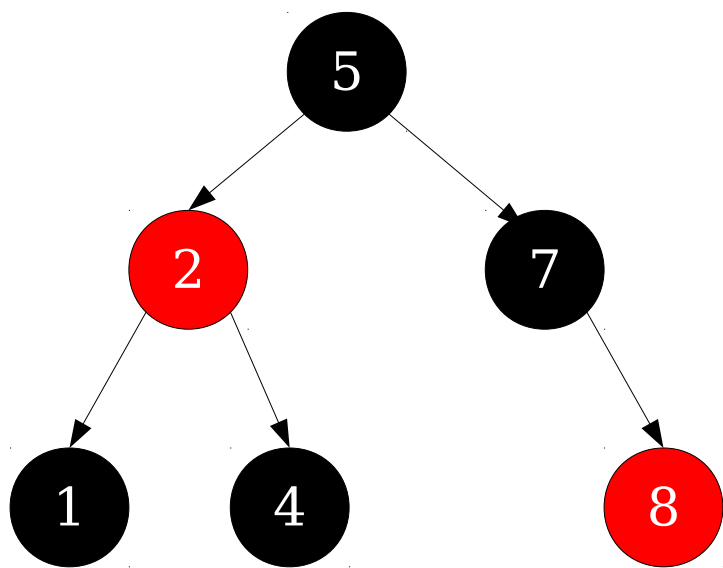
Example of a red-black tree



4. All simple paths from any node x to a descendant leaf have the same number of black nodes = $black-height(x)$.

Are these Red-Black trees? Can they be colored properly?





Red Black Tree Properties

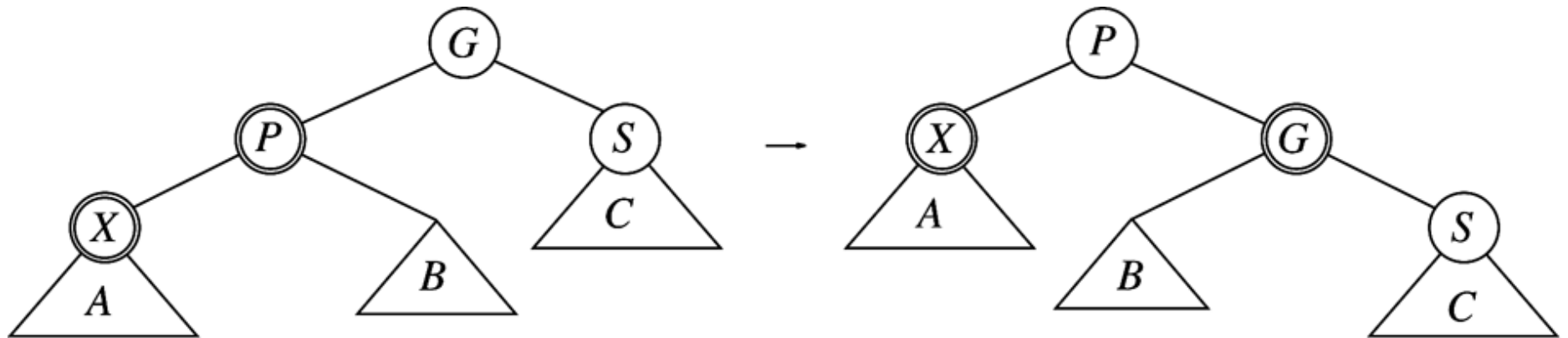
- Node of black height k has height at most $2k$,
- Node of black height k has at least $2^k - 1$ descendants,
- RB Tree with n value nodes has height at most $2 \log(n+1)$
- Proof: (More detail in book...merge red nodes with parent black nodes...)
 - Bottom layer has at least 2^k nodes. If k is black height of node, then max number of nodes in path to leaf is K , min number of nodes n in path to leaf is $K/2$.
 - So, tree is **full** to level $K/2$.
 - Number of nodes n is at least $2^{K/2}$
 - So, $n + 1 \geq 2^{K/2} \Rightarrow L \leq 2 \log_2(n + 1)$

Nice Result for RB Trees

- Normal binary search tree insertions and deletions
- Will result in temporary violation of RB tree properties
- Algorithm restores the RB tree properties using:
 - Recoloring
 - Rotations
- Complexity of insertion and deletion: $O(\log(n))$
- **Bad news:** Many cases to handle —> complicated code
 - Detailed in the textbook
 - We will overview; if you need to implement, look up details for each case

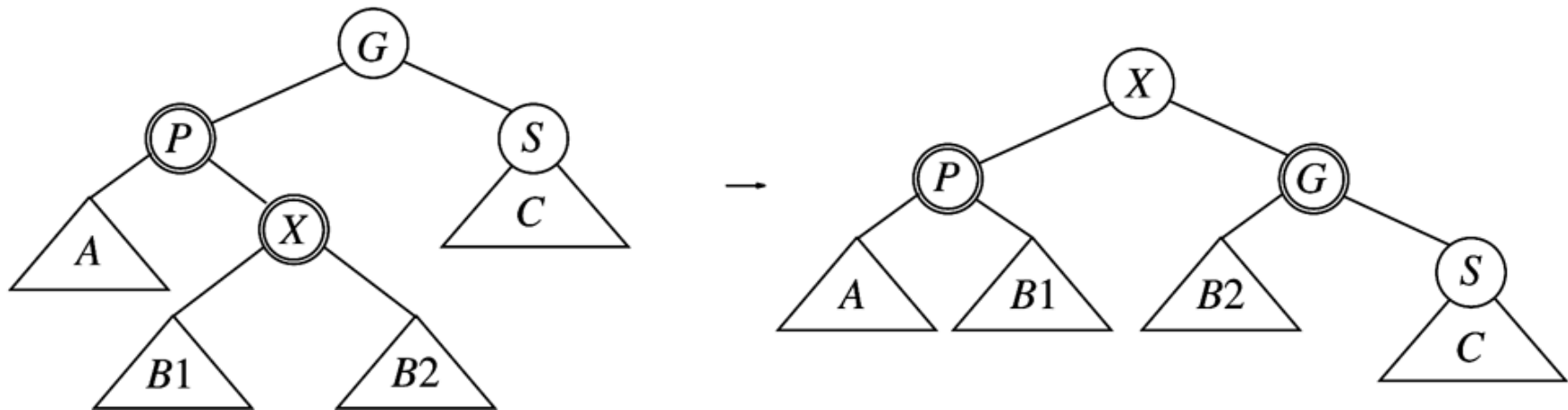
Tree Rotations

Right rotate (Zig) G



Tree Rotations

Left-Right rotate (Zag-Zig): Left rotate P, right rotate G



Insertion in RB Trees

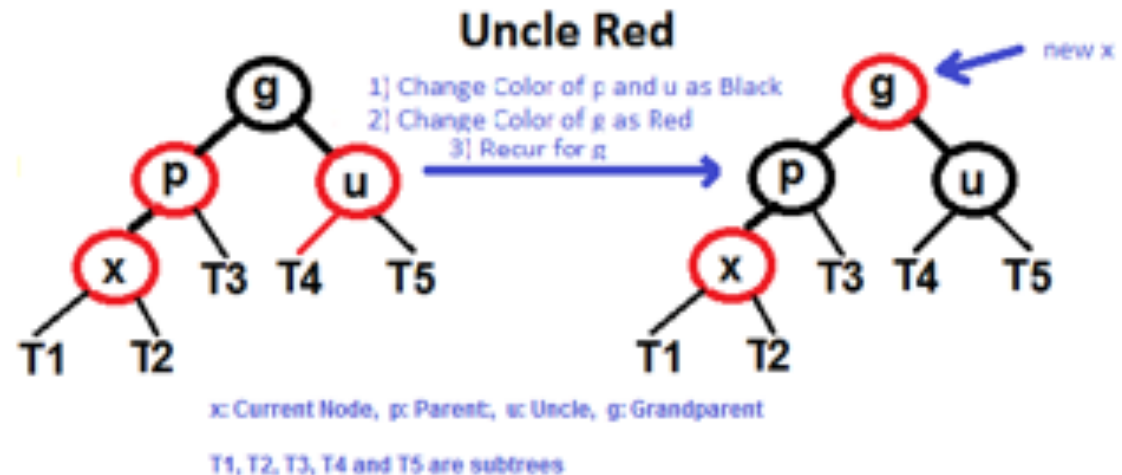
Insert as in Binary Search tree and color the new node Red.

1. If parent of new node is Black, nothing needs to be done.
Tree is RB tree
2. If parent is **RED**, and uncle is also **RED**,

A. Change color of parent **and** uncle as **BLACK**

B. Change color of
grandparent as **RED**

C. Continue fixing up tree
if grandparent has **RED**
parent



Insertion in RB Trees (2)

Insert as in BST tree and color the new node Red.

3. If parent is **RED**, and uncle is **BLACK**, 4 possible cases with parent (p), grandparent (g)

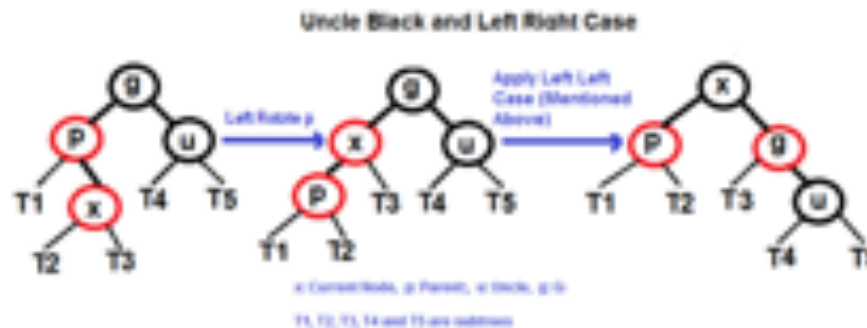
- A. x left of p left of g (Left-left)
- B. x left of p right of g (left-right)
- C. x right of p right of g (right-right) mirror A
- D. x right of p, left of g (right-left) mirror B



CASE A

Complexity: at most 2 rotations, height/2 recoloring
 $O(\log(n))$

CASE D



Deletion in RB Trees

Remove node as in regular binary search trees

Fix resulting BST to restore red-black property

Problem: Many special cases - Discussed in great detail in CLRS

Will illustrate with examples, show they can be done in $O(\log(n))$ time.

Reminder: BST Delete Node X

- X is leaf: Simply delete it
- X has only one child: Copy the child to node X, **delete** the child
- X has two children: Find inorder successor of the node. Copy right child of successor to replace successor. Copy contents of successor to X into X; **delete** inorder successor.

Deletion in RB Trees - 2

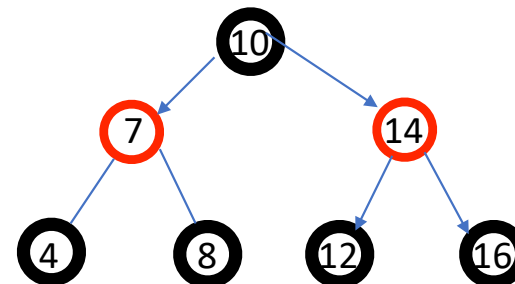
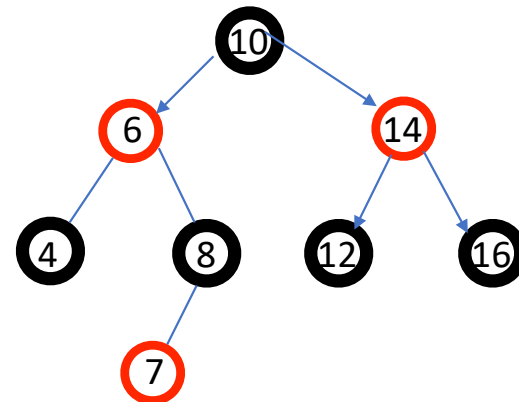
Easy case: Node that will be deleted in BST Remove is RED. Then, just delete it as in BST case and all will be well.

e.g. Remove 7. Simply delete node 7.

e.g Remove 6.

6's data replaced by 7's data, same color

7 is deleted



Deletion in RB Trees - 3

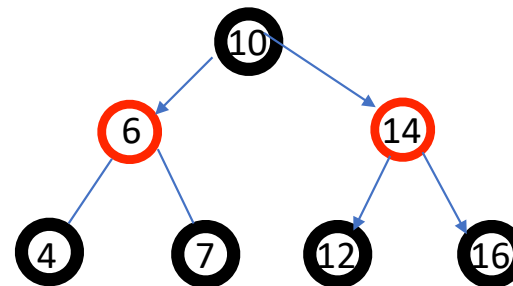
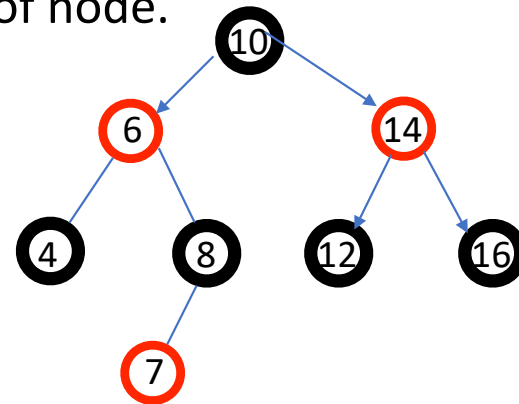
Easy case: Node that will be deleted in BST Remove is black and has single red child. Then, just delete it as in BST case, but keep color of node.

e.g. Remove 8.

Find node to delete as 7.

Copy 7 to 8, keeping original color of 8.

-> Preserves black height.



Deletion in RB Trees - 4

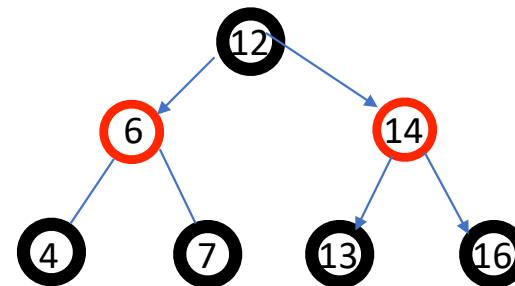
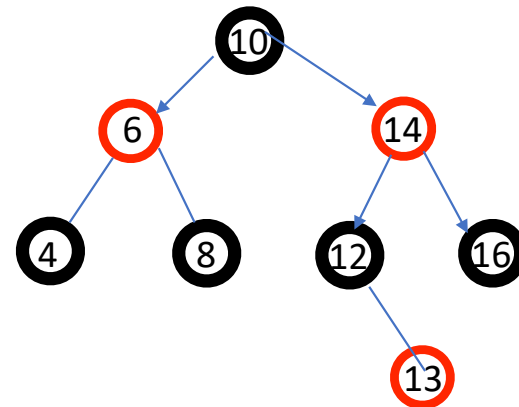
Easy case: Node that will be deleted in BST Remove is black and has single red child. Then, just delete it as in BST case, but keep color of node.

e.g. Remove 10.

Find node to delete as 12.

Copy 12 to 10, keeping original color of 10.

Copy 13 to 12, change color of 13 to 12's color



Deletion in RB Trees - 5

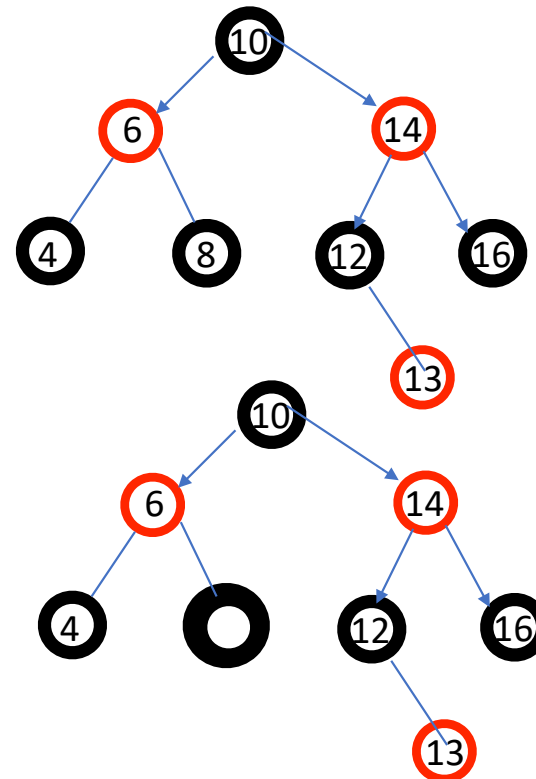
Hard case: Node that will be deleted in BST Remove is black and has two black children! (e.g. Null). This creates black height imbalance

e.g. Remove 8.

Creates an imbalance in black height.

We identify this as a “double black”

Must correct with rotations, recoloring.



Deletion in RB Trees - 6

Hard case: Node that will be deleted in BST Remove is black and has two black children! (e.g. Null). This creates black height imbalance

e.g. Remove 8.

Creates an imbalance in black height.

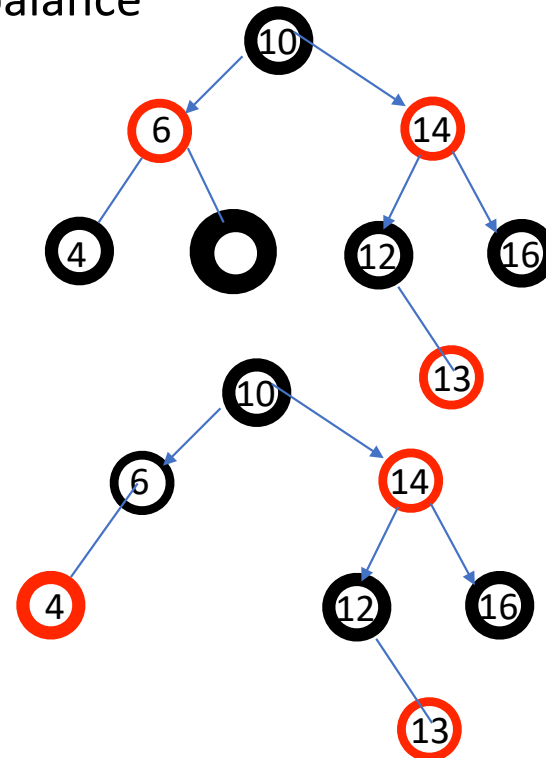
We identify this as a “double black”

Must correct with rotations, recoloring.

Sibling black → recolor as red; if parent red

Recolor as black. If parent is black, color as

“double black” and recur up the tree!



Deletion in RB Trees - 7

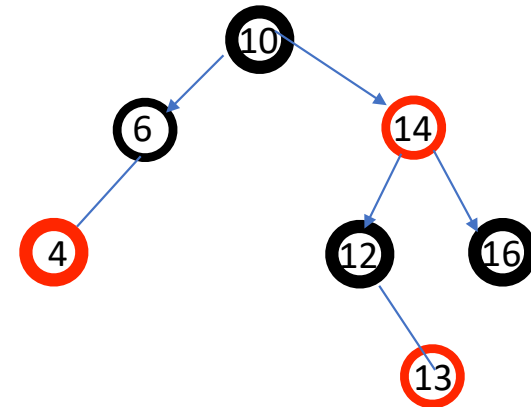
Hard case: Node that will be deleted in BST Remove is black and has two black children! (e.g. Null). This creates black height imbalance

Sibling is black, has red child

e.g. delete 16

a) Rotate 13 left, make it black, make 12 red

b) Rotate 13 right, restoring black height



Deletion in RB Trees - 8

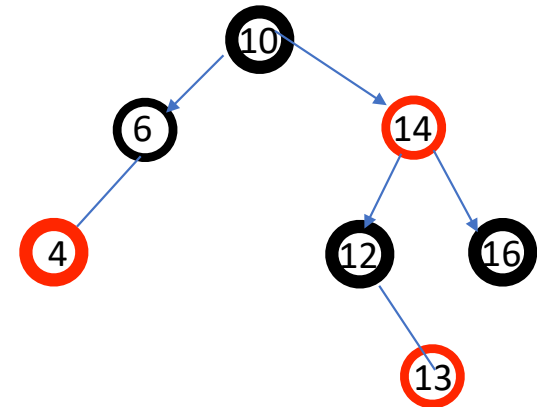
Hard case: Node that will be deleted in BST Remove is black and has two black children! (e.g. Null). This creates black height imbalance

Sibling is red

e.g. delete 6

a) Rotate 14 left, make it black

b) Adds black height automatically



RB Trees Summary

1) Can perform insertion in $O(\log(n))$

2) Can perform deletions in $O(\log(n))$

$O(\log(n))$ recoloring maximum

One or two rotations

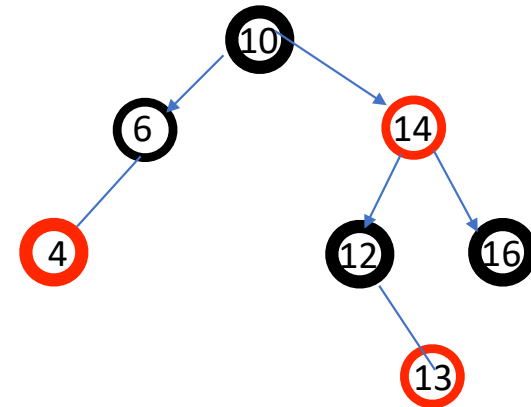
Conclusion: find, insert, delete are $O(\log(n))$

So why are RB trees used more than AVL trees?

AVL trees take more space

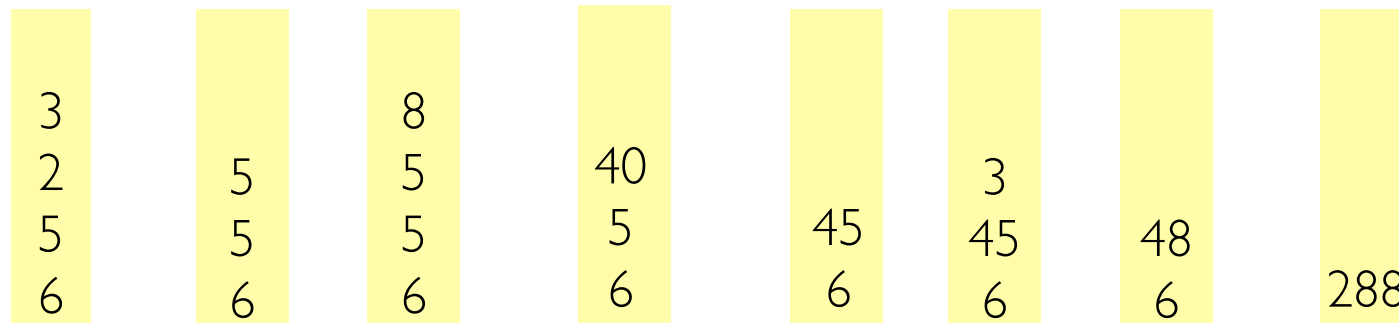
AVL trees do more rotations on insert, delete $\rightarrow$ more overhead

Drawbacks of RB trees: memory overhead to maintain RB property, 1 bit extra



STACK IS FUNDAMENTAL

- Reverse Polish: 6 5 2 3 + 8 * + 3 + *



- See also conversion: infix $\rightarrow$ postfix

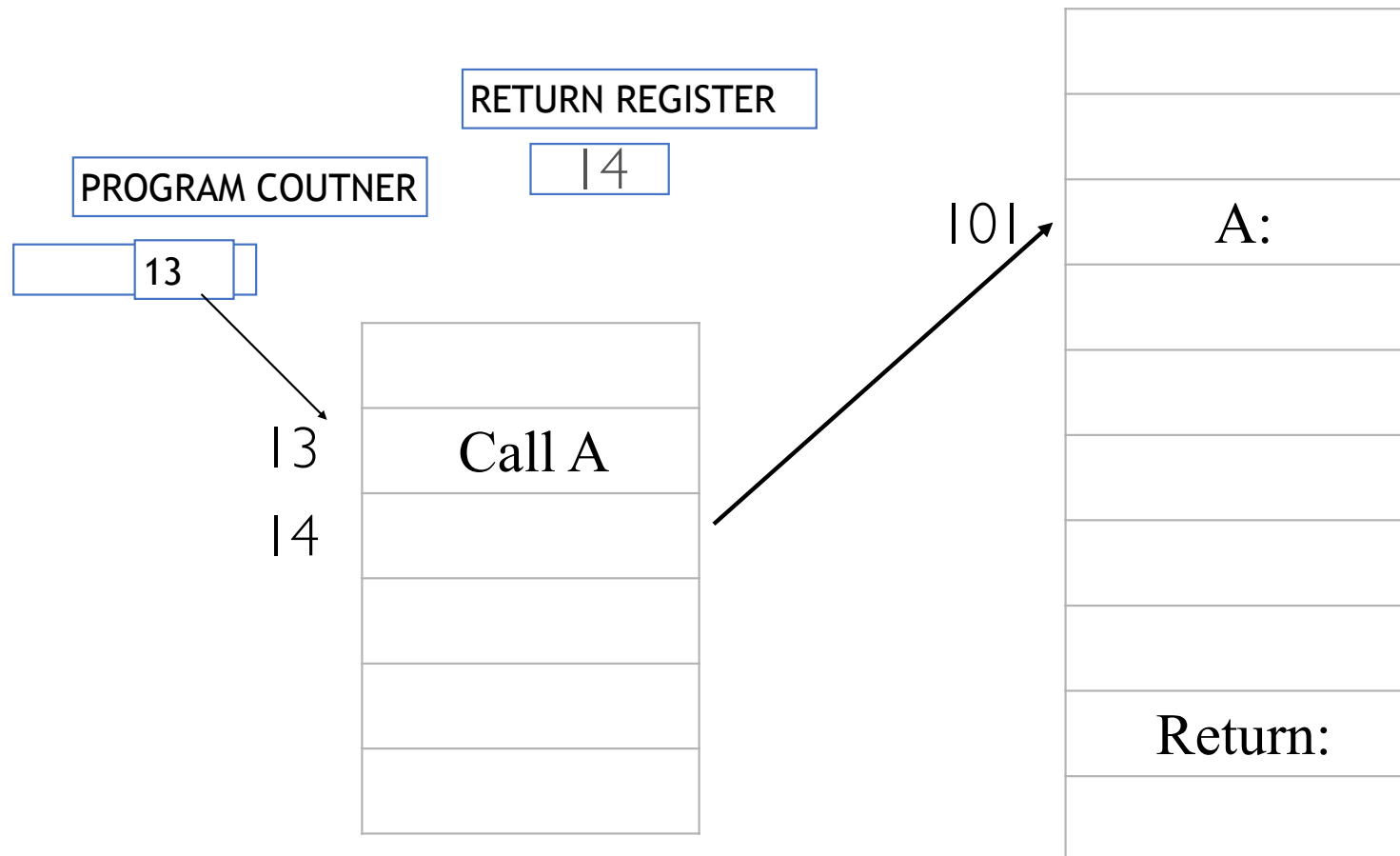
$$(6 * (5 + ((2+3)*8 + 3))) = 6 \ 5 \ 2 \ 3 \ + \ 8 \ * \ + \ 3 \ + \ *$$

- Execution Stacks for Function Calls:
 - Fixed return register
 - First line of subroutine (nested)
 - Execution stack (recursive)

FORTTRAN'S EVOLUTION OF THE SUBROUTINE CALL

- Function Call & Return
- Version 1 -- Return Register
 - ◆ no nesting
- Version 2 --- Return to top of Function
 - ◆ nesting but no recursion
- Version 3 --- The stack frame AT LAST!
 - ◆ Call yourself (recursion)

VERSION I



VERSION 2

PROGRAM COUNTER

13



13

Call A

14

10

A: 14

Call B:

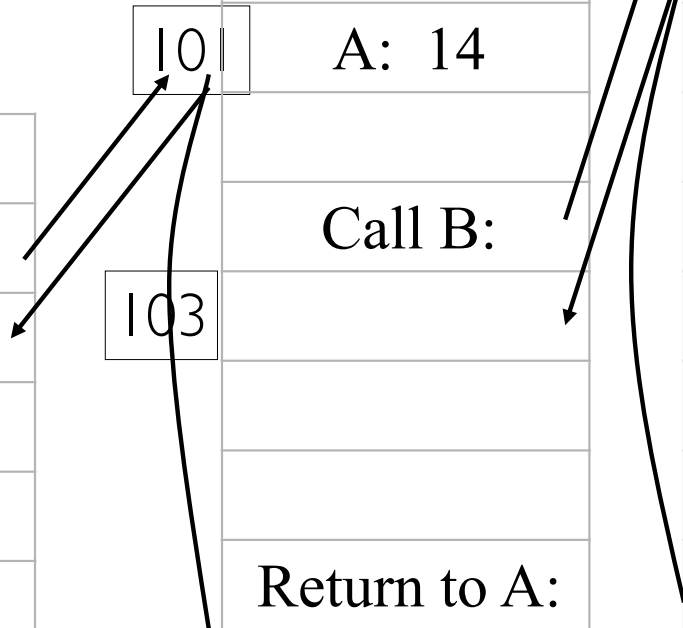
103

Return to A:

202

B: 103

Return to B:



Version 3

PROGRAM COUNTER

13

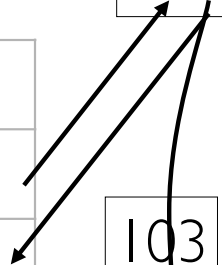


13

Call A

14

101



103

A:

Call A:

Return to A:

STACK

103

Temp

Arg2

Arg1

14

WORST CASE HEIGHT $H(N)$ FOR AVL

- Minimum # of Nodes (see Fig 4.33):

$$N(H) = N(H-1) + N(H-2) + 1 > N(H-1) + N(H-2)$$

- Almost Fibonacci: $F_k = F_{k-1} + F_{k-2}$

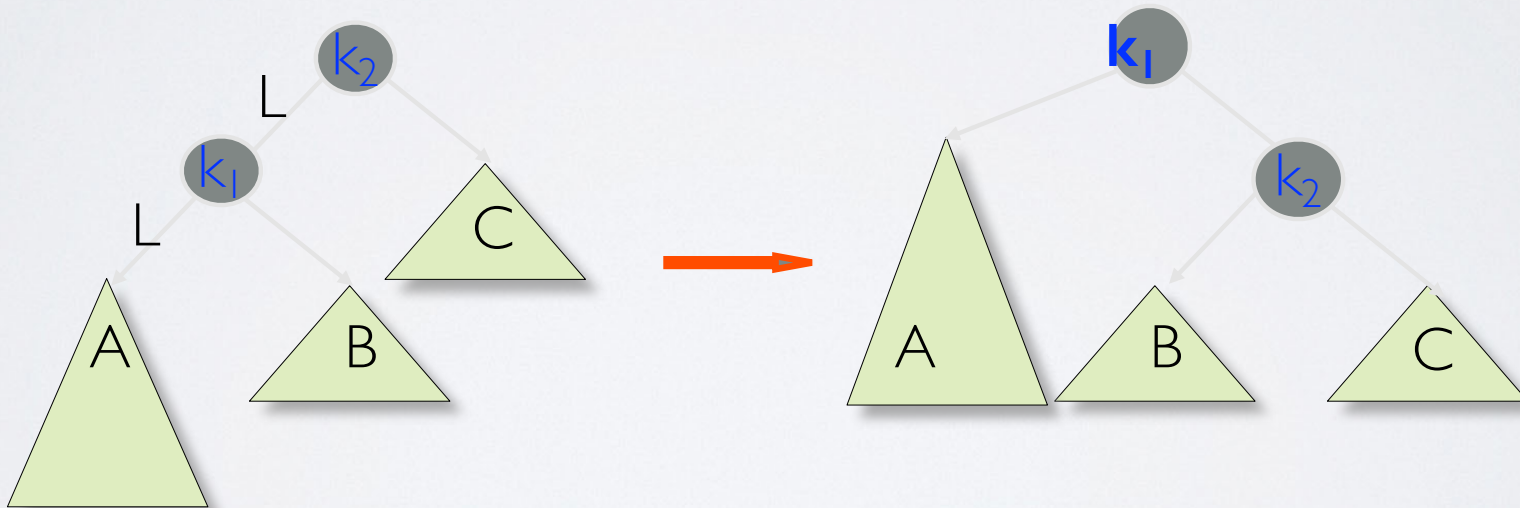
◆ So $N(H) > F_H \sim c^H$ with $c = (1 + 5^{1/2})/2 = 1.618034$

◆ Or $H < \log(N)/\log(c) \quad 1.440420 \log_2(N) = 2.078 \ln(N)$
 $= 4.784 \log_{10}(N)$

(Better estimate: $H = 1.44 \log_2(N+2) - 0.328$

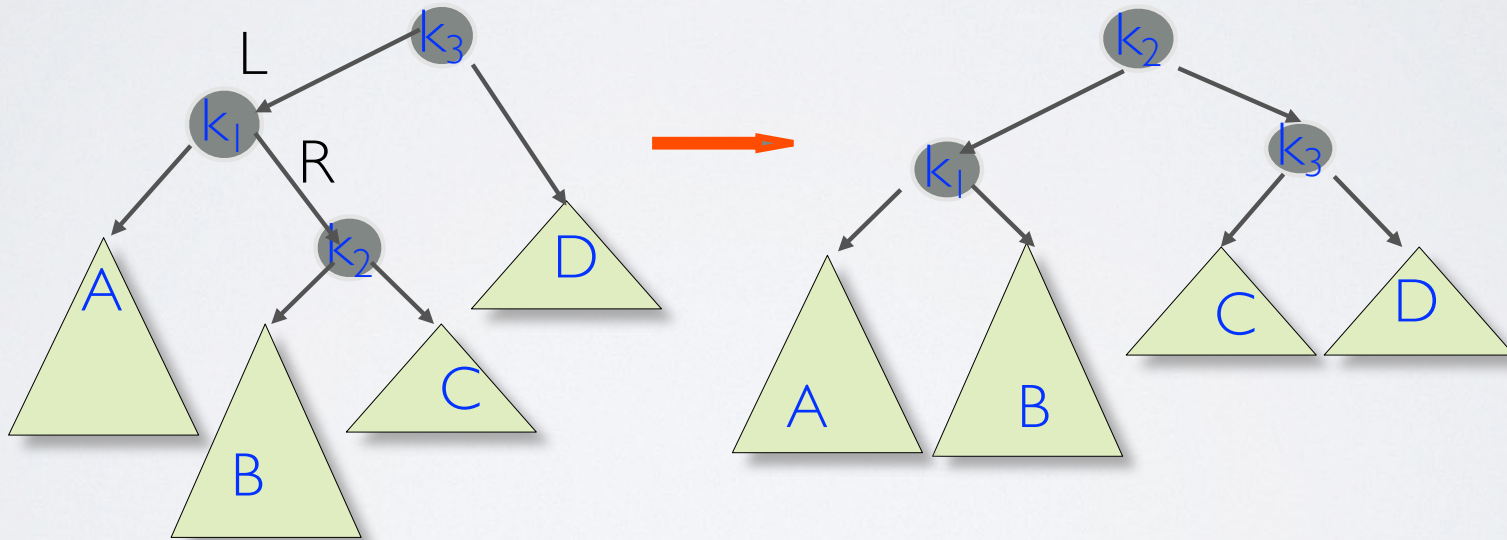
ZIG-ZIG INSERTION FOR LL OR RR:

- Insert New Key along path going **L**eft and **L**eft again into A:
- This cause violation of AVL balance.
- k_2 is lowest node failing AVL balance.
- Single rotation of $k_1 \rightarrow k_2$ restores AVL balance



ZIG-ZAG INSERTION FOR LR

- Insert New Key along path going **L**eft and then **R**ight into B:
- This cause violation of AVL balance.
- k_3 is lowest node failing AVL balance.
- Double rotation of $k_1 \rightarrow k_2 \rightarrow k_3$ restores AVL balance



HUFFMAN CODING

- ❑ Place all letters at leaves of a binary tree
 - ❑ The code is path (i.e. address) of each leaf.
 - ❑ Binary code for each letter: e.g. "a" = 01001, "b" = 101, ..

$$\text{ext. depth} = \sum_i w_i d_i, \quad \text{average code length} = \frac{\sum_i w_i d_i}{\sum_i w_i} = \sum_i p_i d_i$$

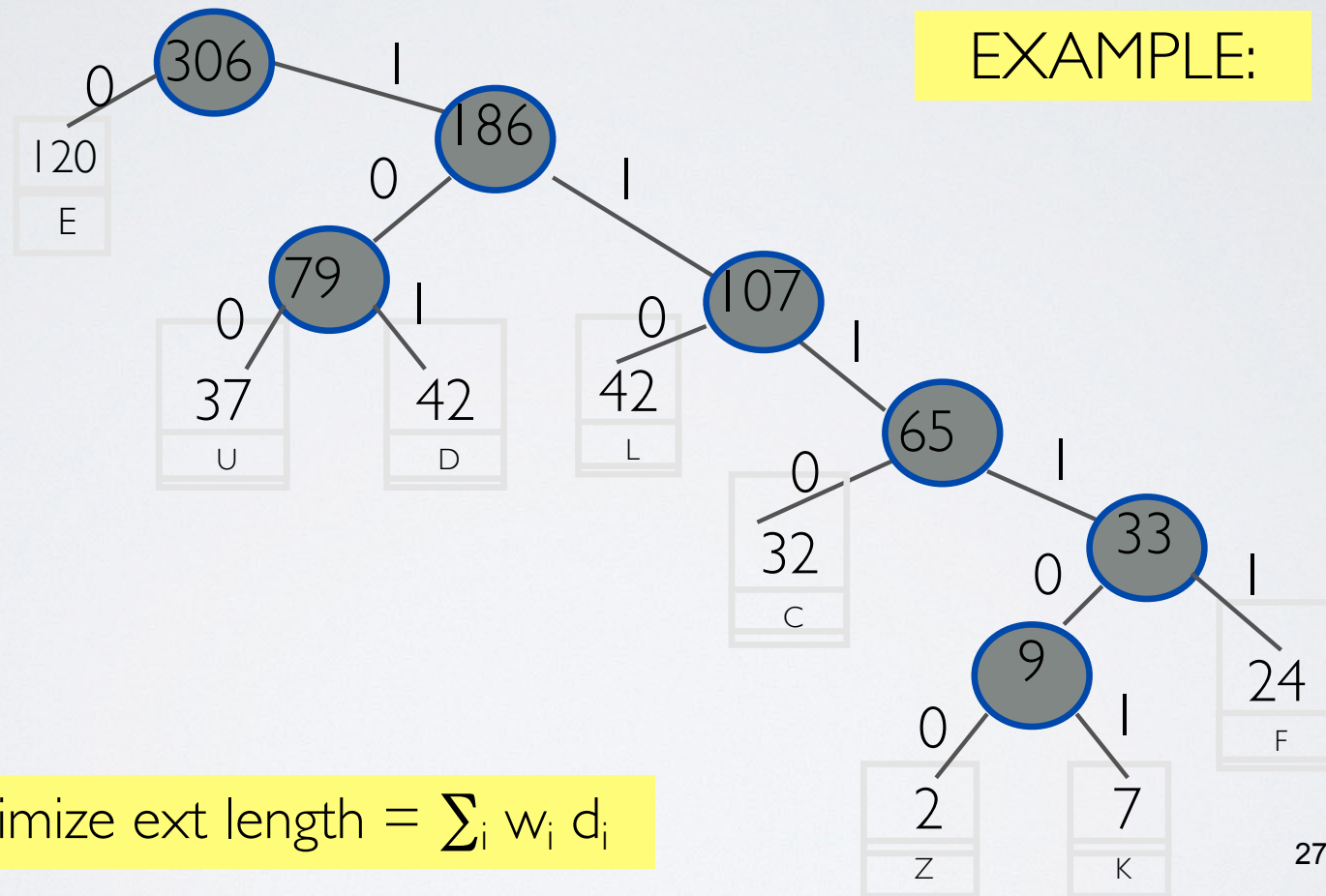
Build the Huffman tree:

- ❑ Sort symbol list: $w_1 < w_2 < \dots < w_N$
- ❑ Remove w_1 and w_2 and place as left and right children of parent $w_{(12)}$
- ❑ Place $w_{(12)} = w_1 + w_2$ in symbol list and Repeat

W_i →

2	7	24	32	37	42	42	120
Z	K	F	C	U	D	L	E

EXAMPLE:



Minimize ext length = $\sum_i w_i d_i$

RESULTING CODE: $\text{AVERAGE BITS/CHAR} = 785/306 = 2.565$

Letter	Weight	Code	Bits	Count
■ C	32	1110	4	128
■ D	42	101	3	126
■ E	120	0	1	120
■ F	24	11111	5	120
■ K	7	111101	6	42
■ L	42	110	3	126
■ U	37	100	3	111
■ Z	2	111100	6	12
Total: 306				785

PROOF BY INDUCTION

- *Base case $N=2$ has minimum with $d_1 = d_2 = 1$*
- *Two smallest weights w_1 & w_2 are at max depth*
- *(If not swap with any other is smaller: See identity next)*
 - ◆ *Can swap to give same parent $w_{12} = w_1 + w_2$*
- *Hence prove for N :*
- *$\text{Min}[(d_{12} + 1) (w_1 + w_2) + w_3 d_3 + \dots + w_N d_N]$ over all trees T*
$$= (w_1 + w_2) + \text{Min}[d_{12} w_{12} + w_3 d_3 + \dots + w_N d_N]$$

“SCHWARTZ” PARING INEQUALITY!

Need for Huffman and Many Opt Algorithms

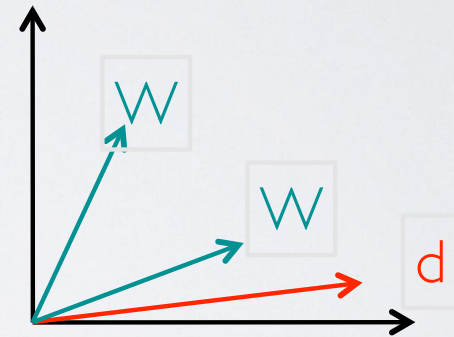
Prove: (more parallel is larger!)

$$w_S d_S + w_L d_L > w_L d_S + w_S d_L$$

because $(w_L - w_S)(d_L - d_S) > 0$

scalar product is larger

when w and d are more nearly parallel!



MORE OPTIMIZATION

- Object Function and elementary move

- Sorting $S = \text{MIN}_{\pi} \sum_i i * a[\pi(i)]$

- swap minimize $i a[i] + j a[j]$ if out of order

- Continuum vs Discrete:

- Bisection : $\text{Log}(N)$ vs error $\Rightarrow$ error/2

- Find zero: $f(x) = 0$ or (continuous)

- Find key $f[i] = (a[i] - \text{key})^2 = 0$ ($a[i]$ sorted)

- Newton's, Secant, Regula falsi

- & Dictionary method (linear extrapolation)

- $\text{Log}(\text{Log}(N))$ vs error $\Rightarrow$ $(\text{error})^\phi$

ϕ is 2 for Newton and the golden ratio 1.618 for secant.